# Chapter 2

# ULP Coprocessor (ULP-FSM, ULP-RISC-V)

## 2.1 Overview

The ULP coprocessor is an ultra-low-power processor that remains powered on when the chip is in Deep-sleep (see Chapter 10 *Low-power Management (RTC_CNTL)*). Hence, users can store in RTC memory a program for the ULP coprocessor to access RTC peripherals, internal sensors, and RTC registers during Deep-sleep.

In power-sensitive scenarios, the main CPU goes to sleep mode to lower power consumption. Meanwhile, the coprocessor is woken up by ULP timer, and then monitors the external environment or interacts with the external circuit by controlling peripherals such as RTC GPIO, RTC I2C, SAR ADC, or temperature sensor (TSENS). The coprocessor wakes the main CPU up once a wakeup condition is reached.

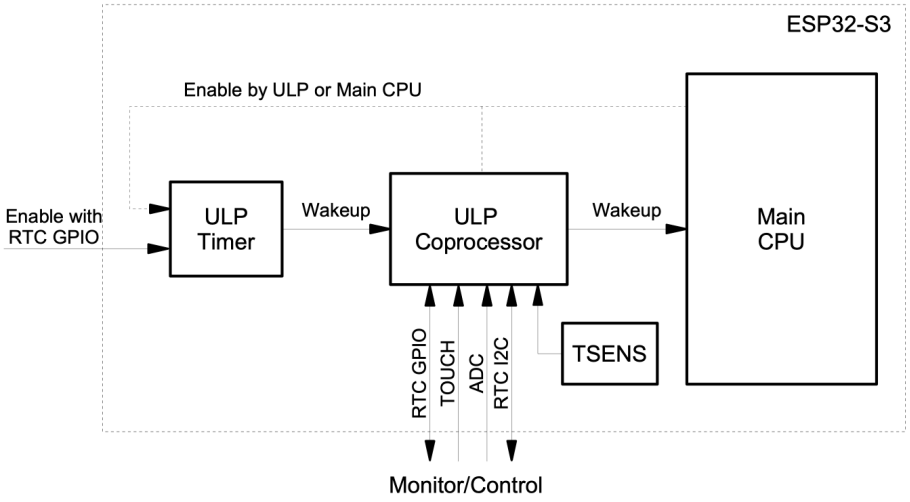



Figure 2.1-1. ULP Coprocessor Overview

ESP32-S3 has two ULP coprocessors, with one based on RISC-V instruction set architecture (ULP-RISC-V) and the other on finite state machine (ULP-FSM). Users can choose between the two coprocessors depending on their needs.

## 2.2 Features

- Access up to 8 KB of SRAM RTC slow memory for instructions and data
- Clocked with 17.5 MHz RTC_FAST_CLK

- Support working in normal mode and in monitor mode
- Wake up the CPU or send an interrupt to the CPU
- Access peripherals, internal sensors and RTC registers

ULP-FSM and ULP-RISC-V can not be used simultaneously. Users can only choose one of them as the ULP coprocessor of ESP32-S3. The differences between the two coprocessors are shown in the table below.

Table 2.2-1. Comparison of the Two Coprocessors

| Feature | | ULP Coprocessors | |
|---|---|---|---|
| | | ULP-FSM | ULP-RISC-V |
| Memory (RTC Slow Memory) | | 8 KB | |
| Work Clock Frequency | | 17.5 MHz | |
| Wakeup Source | | ULP Timer | |
| Work Mode | Normal Mode | Assist the main CPU to complete some tasks after the chip is woken up. | |
| | Monitor Mode | Retrieve data from sensors to monitor environment, when the chip is in sleep. | |
| Control Low-Power Peripherals | | ADC1/ADC2 | |
| | | RTC I2C | |
| | | RTC GPIO | |
| | | Touch Sensors | |
| | | Temperature Sensor | |
| Architecture | | Programmable FSM | RISC-V |
| Development | | Special instruction set | Standard C compiler |

ULP coprocessor can access the modules in RTC domain via RTC registers. In many cases the ULP coprocessor can be a good supplement to, or replacement of, the main CPU, especially for power-sensitive applications. Figure 2.2-1 shows the overall layout of ESP32-S3 coprocessor.

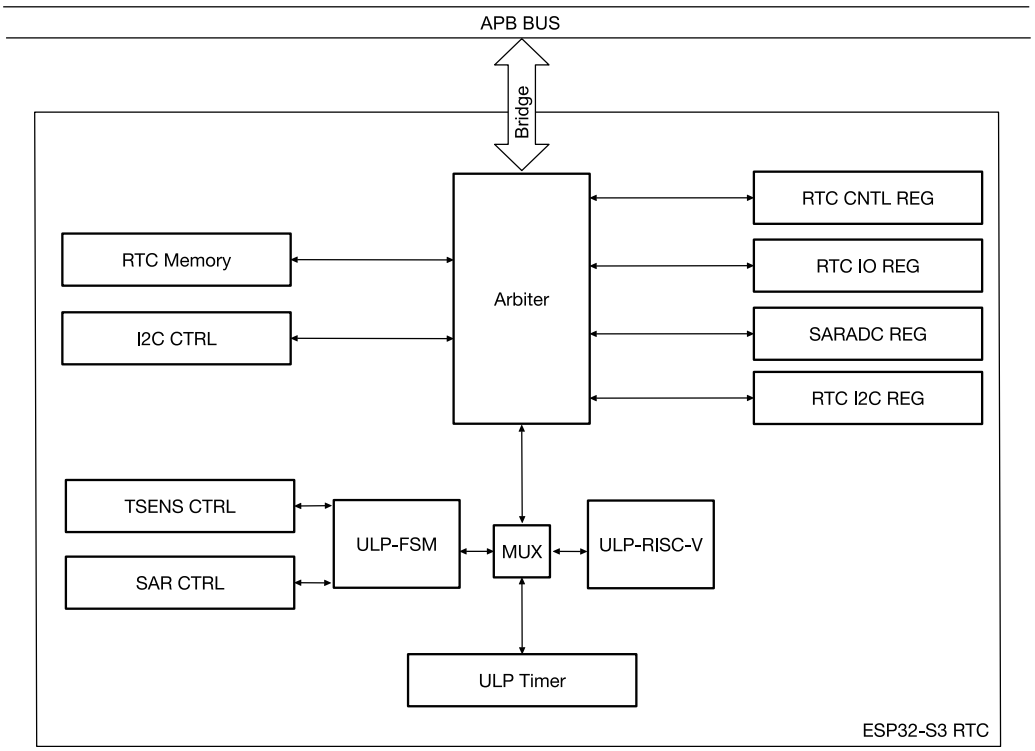



Figure 2.2-1. ULP Coprocessor Diagram

## 2.3 Programming Workflow

The ULP-RISC-V is intended for programming using C language. The program in C is then compiled to RV32IMC standard instruction code. The ULP-FSM is using custom instructions normally not supported by high-level programming language. Users develop their programs using ULP-FSM instructions (see Section 2.5.2).

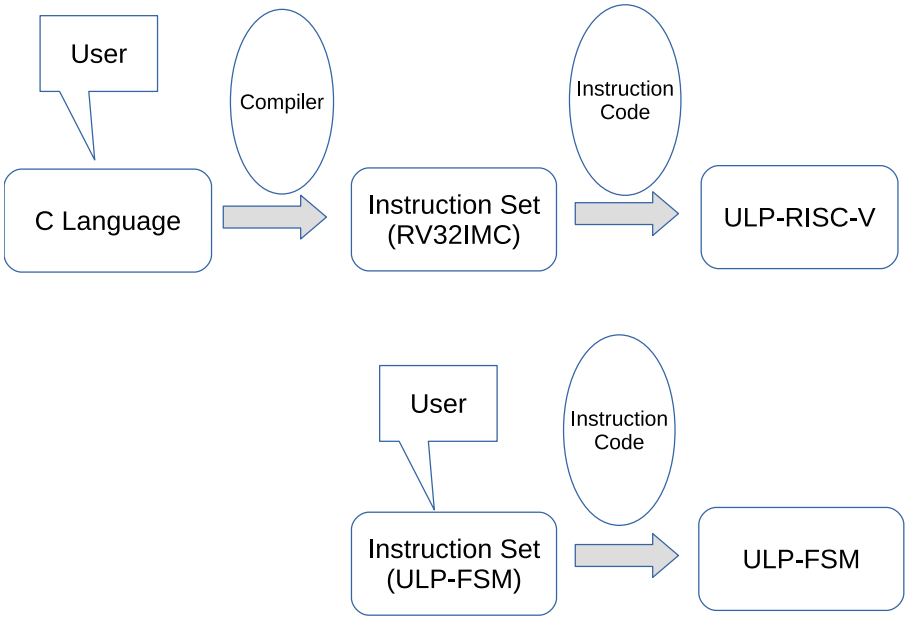



Figure 2.3-1. Programming Workflow

## 2.4 ULP Coprocessor Sleep and Wake-Up Workflow

ULP coprocessor is designed to operate independently of the CPU, while the CPU is either in sleep or running.

In a typical power-saving scenario, the chip goes to Deep-sleep mode to lower power consumption. Before setting the chip to sleep mode, users should complete the following operations.

1. Flash the program to be executed by ULP coprocessor into RTC slow memory.
2. Select the working ULP coprocessor by configuring RTC_CNTL_COCPU_SEL.
   - 0: select ULP-RISC-V
   - 1: select ULP-FSM
3. If ULP-RISC-V is selected as working ULP coprocessor, please
   - set and reset RTC_CNTL_COCPU_CLK_FO;
   - set RTC_CNTL_COCPU_CLKGATE_EN.
4. Set sleep cycles for the timer by configuring RTC_CNTL_ULP_CP_TIMER_1_REG.
5. Enable the timer by software or by RTC GPIO;
   - By software: set RTC_CNTL_ULP_CP_SLP_TIMER_EN.

- By RTC GPIO: set RTC_CNTL_ULP_CP_GPIO_WAKEUP_ENA. For more information, see Chapter 10 *Low-power Management (RTC_CNTL)*.

6. Set the system into sleep mode.

When the system is in Deep-sleep mode:

1. The timer periodically sets the low-power controller (see Chapter 10 *Low-power Management (RTC_CNTL)*) to Monitor mode and then wakes up the coprocessor.
2. Coprocessor executes some necessary operations, such as monitoring external environment via low-power sensors.
3. After the operations are finished, the system goes back to Deep-sleep mode.
4. ULP coprocessor goes back to halt mode and waits for next wakeup.

In monitor mode, ULP coprocessor is woken up and goes to halt as shown in Figure 2.4-1.

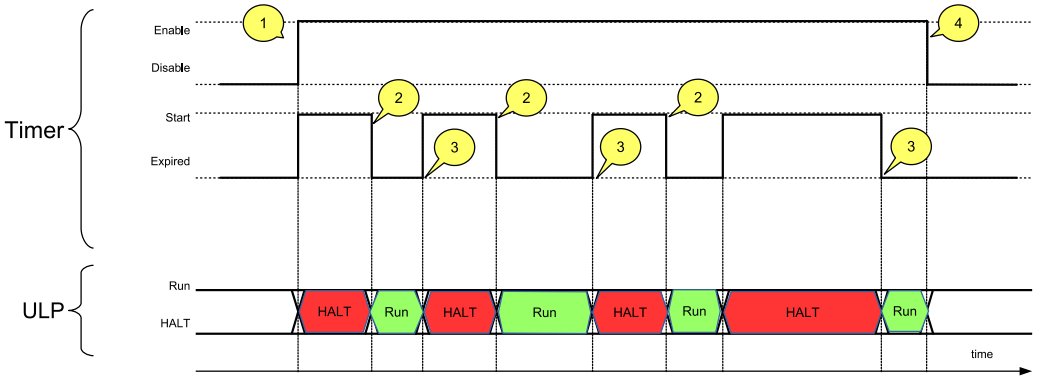



Figure 2.4-1. ULP Sleep and Wakeup Sequence

1. Enable the timer and the timer starts counting.
2. The timer expires and wakes up the ULP coprocessor. ULP coprocessor starts running and executes the program flashed in RTC slow memory.
3. ULP coprocessor goes to halt and the timer starts counting again.
   - Put ULP-RISC-V into HALT: set RTC_CNTL_COCPU_DONE.
   - Put ULP-FSM into HALT: execute HALT instruction.
4. Disable the timer by ULP program or by software. ULP coprocessor exits from monitor mode.
   - Disabled by software: clear RTC_CNTL_ULP_CP_SLP_TIMER_EN.
   - Disabled by RTC GPIO: clear RTC_CNTL_ULP_CP_GPIO_WAKEUP_ENA and set RTC_CNTL_ULP_CP_GPIO_WAKEUP_CLR.

**Note:**

- If the timer is enabled by software (RTC GPIO), it should be disabled by software (RTC GPIO).
- Before setting ULP-RISC-V to HALT, users should configure RTC_CNTL_COCPU_DONE first, therefore, it is recommended to end the flashed program with the following pattern:
  - Set RTC_CNTL_COCPU_DONE to end the operation of ULP-RISC-V and put it into halt;

– Set RTC_CNTL_COCPU_SHUT_RESET_EN to reset ULP-RISC-V.

Enough time is reserved for the ULP-RISC-V to complete the operations above before it goes to halt.

Figure 2.4-2 shows the relationship between the signals and register bits.

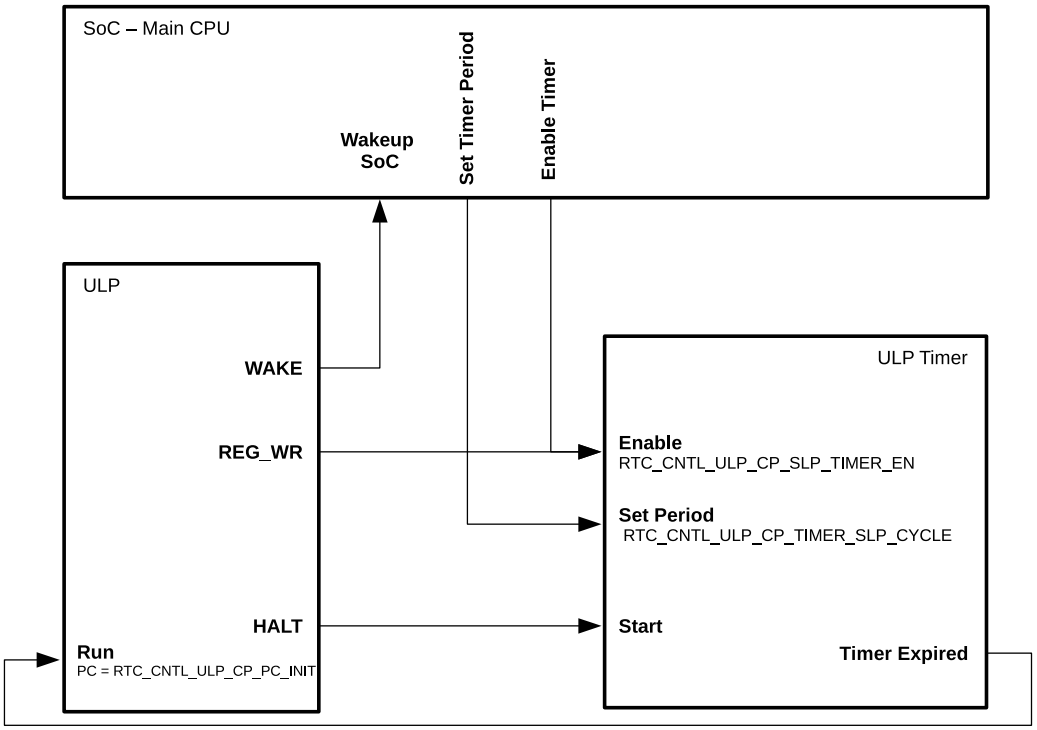



Figure 2.4-2. Control of ULP Program Execution

# 2.5 ULP-FSM

## 2.5.1 Features

ULP-FSM is a programmable finite state machine that can work while the main CPU is in Deep-sleep. ULP-FSM supports instructions for complex logic and arithmetic operations, and also provides dedicated instructions for RTC controllers or peripherals. ULP-FSM can access up to 8 KB of SRAM RTC slow memory (accessible by the CPU) for instructions and data. Hence, such memory is usually used to store instructions and share data between the ULP coprocessor and the CPU. ULP-FSM can be stopped by running HALT instruction.

ULP-FSM has the following features.

- Provides four 16-bit general-purpose registers (R0, R1, R2, and R3) for manipulating data and accessing memory.
- Provides one 8-bit stage count register (Stage_cnt) which can be manipulated by ALU and used in JUMP instructions.
- Supports built-in instructions specially for direct control of low-power peripherals, such as SAR ADC and temperature sensor.

## 2.5.2 Instruction Set

ULP-FSM supports the following instructions.

- ALU: perform arithmetic and logic operations
- LD, ST, REG_RD and REG_WR: load and store data
- JUMP: jump to a certain address
- WAIT/HALT: manage program execution
- WAKE: wake up CPU or communicate with CPU
- TSENS and ADC: take measurements

Figure 2.5-1 shows the format of ULP-FSM instructions.

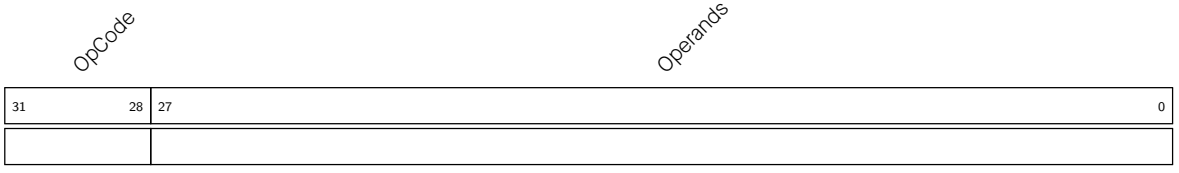



Figure 2.5-1. ULP-FSM Instruction Format

An instruction, which has one OpCode, can perform various operations, depending on the setting of Operands bits. A good example is the ALU instruction, which is able to perform 10 arithmetic and logic operations; or the JUMP instruction, which may be conditional or unconditional, absolute or relative.

Each instruction has a fixed width of 32 bits. A series of instructions can make a program be executed by the coprocessor. The execution flow inside the program uses 32-bit addressing. The program is stored in a dedicated region called Slow Memory, which is visible to the main CPU under an address range of 0x5000_0000 to 0x5000_1FFF (8 KB).

#### 2.5.2.1 ALU - Perform Arithmetic and Logic Operations

ALU (Arithmetic and Logic Unit) performs arithmetic and logic operations on values stored in ULP coprocessor registers, and on immediate values stored in the instruction itself. The following operations are supported.

- Arithmetic: ADD and SUB
- Logic: bitwise logical AND and bitwise logical OR
- Bit shifting: LSH and RSH
- Moving data to register: MOVE
- PC register operations - STAGE_RST, STAGE_INC, and STAGE_DEC

The ALU instruction, which has one OpCode (7), can perform various arithmetic and logic operations, depending on the setting of the instruction bits [27:21].

##### Operations Among Registers

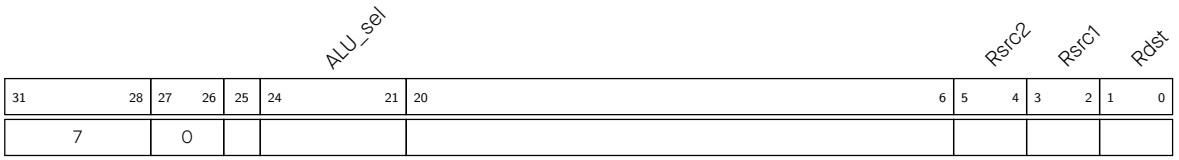



Figure 2.5-2. Instruction Type — ALU for Operations Among Registers

When bits [27:26] of the instruction in Figure 2.5-2 are set to 0, ALU performs operations on the data stored in ULP-FSM registers R[0-3]. The types of operations depend on the setting of the instruction bits ALU_sel[24:21] presented in Table 2.5-1.

| **Operand** | **Description** - see Figure 2.5-2 |
|---|---|
| *Rdst* | Register R[0-3], destination |
| *Rsrc1* | Register R[0-3], source |
| *Rsrc2* | Register R[0-3], source |
| *ALU_sel* | ALU operation selection, see Table 2.5-1 |

| ALU_sel | Instruction | Operation | Description |
|---|---|---|---|
| 0 | ADD | *Rdst = Rsrc1 + Rsrc2* | Add to register |
| 1 | SUB | *Rdst = Rsrc1 - Rsrc2* | Subtract from register |
| 2 | AND | *Rdst = Rsrc1 & Rsrc2* | Bitwise logical AND of two operands |
| 3 | OR | *Rdst = Rsrc1 \| Rsrc2* | Bitwise logical OR of two operands |
| 4 | MOVE | *Rdst = Rsrc1* | Move to register |
| 5 | LSH | *Rdst = Rsrc1 << Rsrc2* | Bit shifting left |
| 6 | RSH | *Rdst = Rsrc1 >> Rsrc2* | Bit shifting right |

Table 2.5-1. ALU Operations Among Registers

**Note:**

- ADD or SUB operations can be used to set or clear the overflow flag in ALU.
- All ALU operations can be used to set or clear the zero flag in ALU.

### Operations with Immediate Value

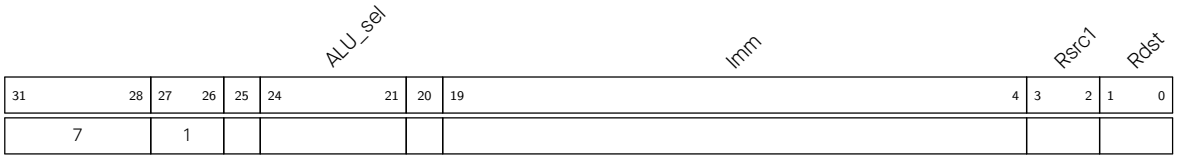



Figure 2.5-3. Instruction Type — ALU for Operations with Immediate Value

When bits [27:26] of the instruction in Figure 2.5-3 are set to 1, ALU performs operations using register R[0-3] and the immediate value stored in instruction bits [19:4]. The types of operations depend on the setting of the instruction bits ALU_sel[24:21] presented in Table 2.5-2.

| **Operand** | **Description** - see Figure 2.5-3 |
|---|---|
| *Rdst* | Register R[0-3], destination |
| *Rsrc1* | Register R[0-3], source |
| *Imm* | 16-bit signed immediate value |
| *ALU_sel* | ALU operation selection, see Table 2.5-2 |

| ALU_sel | Instruction | Operation | Description |
|---|---|---|---|
| 0 | ADD | *Rdst* = *Rsrc1* + *Imm* | Add to register |
| 1 | SUB | *Rdst* = *Rsrc1* - *Imm* | Subtract from register |
| 2 | AND | *Rdst* = *Rsrc1* & *Imm* | Bitwise logical AND of two operands |
| 3 | OR | *Rdst* = *Rsrc1* \| *Imm* | Bitwise logical OR of two operands |
| 4 | MOVE | *Rdst* = *Imm* | Move to register |
| 5 | LSH | *Rdst* = *Rsrc1* << *Imm* | Bit shifting left |
| 6 | RSH | *Rdst* = *Rsrc1* >> *Imm* | Bit shifting right |

Table 2.5-2. ALU Operations with Immediate Value

**Note:**

- ADD or SUB operations can be used to set or clear the overflow flag in ALU.
- All ALU operations can be used to set or clear the zero flag in ALU.

### Operations with Stage Count Register

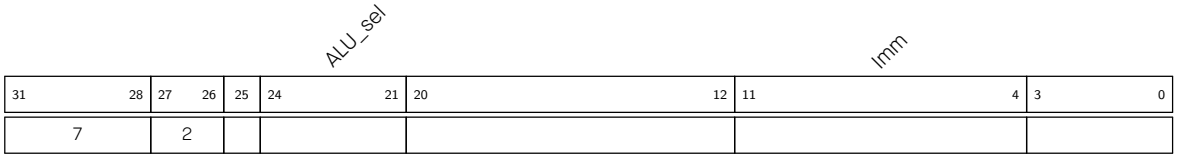



Figure 2.5-4. Instruction Type — ALU for Operations with Stage Count Register

ALU is also able to increment or decrement by a given value, or reset the 8-bit register Stage_cnt. To do so, bits [27:26] of instruction in Figure 2.5-4 should be set to 2. The type of operation depends on the setting of the instruction bits ALU_sel[24:21] presented in Table 2.5-4. The Stage_cnt is a separate register and is not a part of the instruction in Figure 2.5-4.

| Operand | Description - see Figure 2.5-4 |
|---|---|
| *Imm* | 8-bit signed immediate value |
| *ALU_sel* | ALU operation selection, see Table 2.5-3 |
| *Stage_cnt* | Stage count register, a 8-bit separate register used to store variables, such as loop index |

| ALU_sel | Instruction | Operation | Description |
|---|---|---|---|
| 0 | STAGE_INC | *Stage_cnt* = *Stage_cnt* + *Imm* | Increment stage count register |
| 1 | STAGE_DEC | *Stage_cnt* = *Stage_cnt* - *Imm* | Decrement stage count register |
| 2 | STAGE_RST | *Stage_cnt* = 0 | Reset stage count register |

Table 2.5-3. ALU Operations with Stage Count Register

**Note:** This instruction is mainly used with JUMPS instruction based on the stage count register to form a stage count for-loop. For the usage, please refer to the following pseudocode:

*STAGE_RST* // clear stage count register
*STAGE_INC* // stage count register ++
*{...}* // loop body, containing $n$ instructions
*JUMPS (step = $n$, cond = 0, threshold = $m$)* // If the value of stage count register is less than $m$, then jump to STAGE_INC, otherwise jump out of the loop. By such way, a cumulative for-loop with threshold $m$ is implemented.

### 2.5.2.2 ST – Store Data in Memory

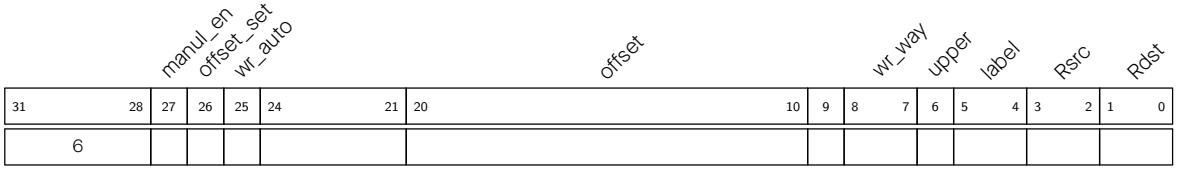



Figure 2.5-5. Instruction Type - ST

| Operand | Description - see Figure 2.5-5 |
|---|---|
| *Rdst* | Register R[0-3], address of the destination, expressed in 32-bit words |
| *Rsrc* | Register R[0-3], 16-bit value to store |
| *label* | Data label, 2-bit user defined unsigned value |
| *upper* | 0: write the low half-word; 1: write the high half-word |
| *wr_way* | 0: write the full-word; 1: with the label; 3: without the label |
| *offset* | 11-bit signed value, expressed in 32-bit words |
| *wr_auto* | Enable automatic storage mode |
| *offset_set* | Offset enable bit.<br>0: Do not configure the offset for automatic storage mode.<br>1: Configure the offset for automatic storage mode. |
| *manul_en* | Enable manual storage mode |

#### Automatic Storage Mode

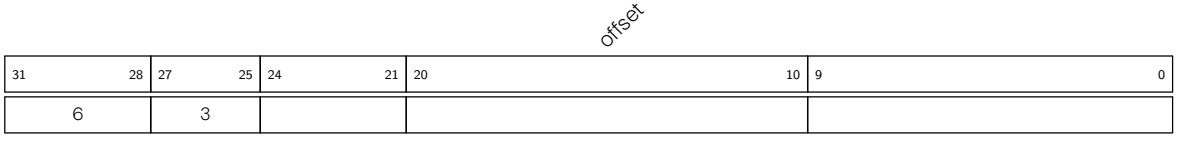


Figure 2.5-6. Instruction Type - Offset in Automatic Storage Mode (ST-OFFSET)

| Operand | Description - see Figure 2.5-6 |
|---|---|
| *offset* | Initial address offset, 11-bit signed value, expressed in 32-bit words |

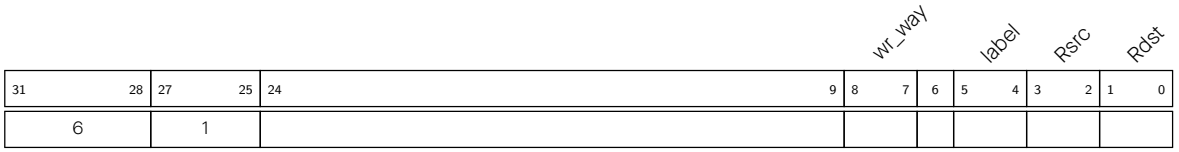



Figure 2.5-7. Instruction Type - Data Storage in Automatic Storage Mode (ST-AUTO-DATA)

| Operand | Description - See Figure 2.5-7 |
|---|---|
| *Rdst* | Register R[0-3], address of the destination, expressed in 32-bit words |
| *Rsrc* | Register R[0-3], 16-bit value to store |
| *label* | Data label, 2-bit user defined unsigned value |
| *wr_way* | 0: write the full-word; 1: with the label; 3: without the label |

**Description**

This mode is used to access continuous addresses. Before using this mode for the first time, please configure the initial address using ST-OFFSET instruction. Executing the instruction ST-AUTO-DATA will store the 16-bit data in *Rsrc* into the memory address *Rdst* + *Offset*, see Table 2.5-4. Write_cnt here indicates the times of the instruction ST-AUTO-DATA executed.

| wr_way | write_cnt | Store Data | Operation |
|---|---|---|---|
| 0 | * | Mem [Rdst + Offset]{31:0} = {PC[10:0],3'b0,Label[1:0],Rsrc[15:0]} | Write full-word, including the pointer and the data |
| 1 | odd | Mem [Rdst + Offset]{15:0} = {Label[1:0],Rsrc[13:0]} | Store the data with label in the low half-word |
| 1 | even | Mem [Rdst + Offset]{31:16} = {Label[1:0],Rscr[13:0]} | Store the data with label in the high half-word |
| 3 | odd | Mem [Rdst + Offset]{15:0} = Rsrc[15:0]} | Store the data without label in the low half-word |
| 3 | even | Mem [Rdst + Offset]{31:16} = Rsrc[15:0] | Store the data without label in the high half-word |

Table 2.5-4. Data Storage Type - Automatic Storage Mode

The full-word written to RTC memory is built as follows:

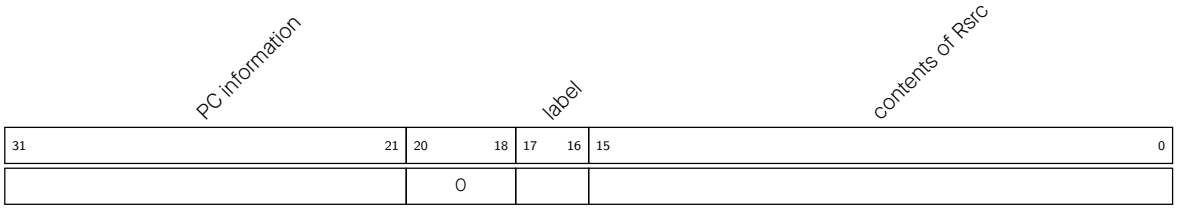



Figure 2.5-8. Data Structure of RTC_SLOW_MEM[Rdst + Offset]

| Bits | Description - See Figure 2.5-8 |
|---|---|
| *bits [15:0]* | store the content of Rsrc |
| *bits [17:16]* | data label, 2-bit user defined unsigned value |
| *bits [20:18]* | 3'b0 by default |
| *bits [31:21]* | hold the PC of current instruction, expressed in 32-bit words |

**Note:**

- When full-word is written, the offset will be automatically incremented by 1 after each ST-AUTO-DATA execution.
- When half-word is written (low half-word first), the offset will be automatically incremented by 1 after twice ST-AUTO-DATA execution.
- This instruction can only access 32-bit memory words.
- The "Mem" written is the RTC_SLOW_MEM memory. Address 0, as seen by the ULP coprocessor, corresponds to address 0x50000000, as seen by the main CPU.

### Manual Storage Mode

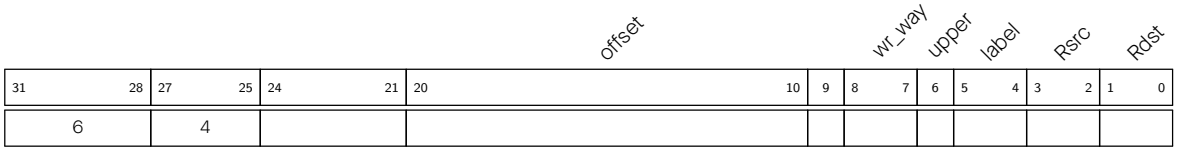



Figure 2.5-9. Instruction Type - Data Storage in Manual Storage Mode

| Operand | Description - See Figure 2.5-9 |
|---|---|
| *Rdst* | Register R[0-3], address of the destination, expressed in 32-bit words |
| *Rsrc* | Register R[0-3], 16-bit value to store |
| *label* | Data label, 2-bit user defined unsigned value |
| *upper* | 0: Write the low half-word; 1: write the high half-word |
| *wr_way* | 0: Write the full-word; 1: with the label; 3: without the label |
| *offset* | 11-bit signed value, expressed in 32-bit words |

**Description**

Manual storage mode is mainly used for storing data into discontinuous addresses. Each instruction needs a storage address and offset. The detailed storage methods are shown in Table 2.5-5.

| wr_way | upper | Data | Operation |
|---|---|---|---|
| 0 | * | Mem [Rdst + Offset]{31:0} = {PC[10:0],3'b0, Label[1:0],Rsrc[15:0]} | Write full-word, including the pointer and the data |
| 1 | 0 | Mem [Rdst + Offset]{15:0} = {Label[1:0],Rsrc[13:0]} | Store the data with label in the low half-word |
| 1 | 1 | Mem [Rdst + Offset]{31:16} = {Label[1:0],Rsrc[13:0]} | Store the data with label in the high half-word |
| 3 | 0 | Mem [Rdst + Offset]{15:0} = Rsrc[15:0] | Store the data without label in the low half-word |
| 3 | 1 | Mem [Rdst + Offset]{31:16} = Rsrc[15:0] | Store the data without label in the high half-word |

Table 2.5-5. Data Storage - Manual Storage Mode

### 2.5.2.3 LD – Load Data from Memory

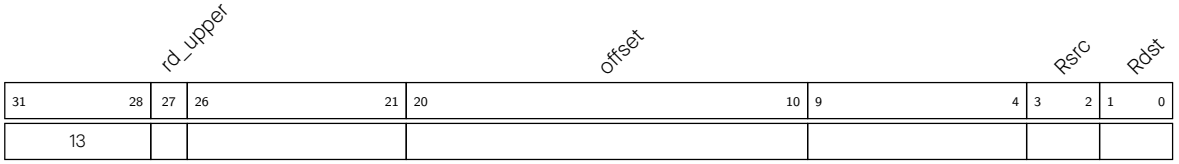



Figure 2.5-10. Instruction Type - LD

| Operand | Description - see Figure 2.5-10 |
|---|---|
| *Rdst* | Register R[0-3], destination |
| *Rsrc* | Register R[0-3], address of destination memory, expressed in 32-bit words |
| *Offset* | 11-bit signed value, expressed in 32-bit words |
| *rd_upper* | Choose which half-word to read:<br>1 - read the high half-word<br>0 - read the low half-word |

**Description**

This instruction loads the low or high 16-bit half-word, depending on *rd_upper*, from memory with address *Rsrc* + *offset* into the destination register *Rdst*:

Rdst[15:0] = Mem[Rsrc + Offset]

**Note:**

- This instruction can only access 32-bit memory words.
- The "Mem" loaded is the RTC_SLOW_MEM memory. Address 0, as seen by the ULP coprocessor, corresponds to address 0x50000000, as seen by the main CPU.

#### 2.5.2.4 JUMP – Jump to an Absolute Address

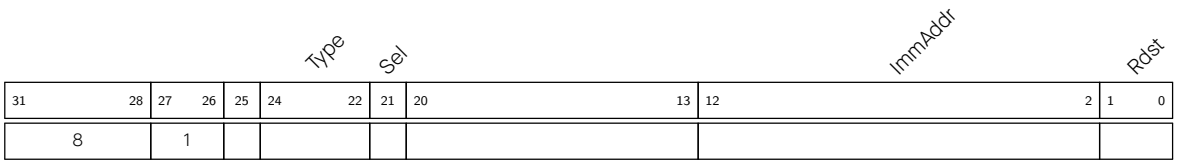



**Figure 2.5-11. Instruction Type - JUMP**

| **Operand** | **Description** - see Figure 2.5-11 |
|---|---|
| *Rdst* | Register R[0-3], containing address to jump to (expressed in 32-bit words) |
| *ImmAddr* | 11-bit address, expressed in 32-bit words |
| *Sel* | Select the address to jump to:<br>0 - jump to the address stored in *ImmAddr*<br>1 - jump to the address stored in *Rdst* |
| *Type* | Jump type:<br>0 - make an unconditional jump<br>1 - jump only if the last ALU operation has set zero flag<br>2 - jump only if the last ALU operation has set overflow flag |

**Note:**

All jump addresses are expressed in 32-bit words.

**Description**

The instruction executes a jump to a specified address. The jump can be either unconditional or based on the ALU flag.

#### 2.5.2.5 JUMPR – Jump to a Relative Address (Conditional upon R0)

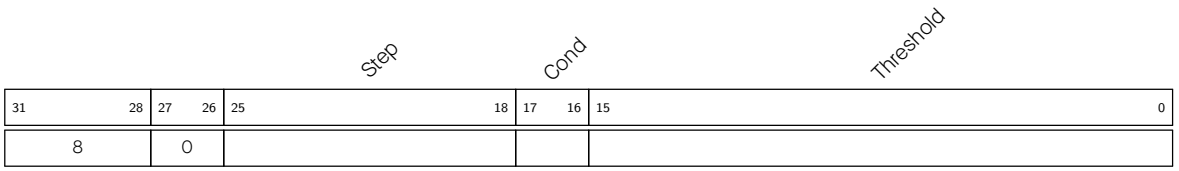



**Figure 2.5-12. Instruction Type - JUMPR**

| Operand | **Description** - see Figure 2.5-12 |
|---|---|
| *Threshold* | Threshold value for condition (see *Cond* below) to jump |
| *Cond* | Condition to jump:<br>0 - jump if R0 < *Threshold*<br>1 - jump if R0 > *Threshold*<br>2 - jump if R0 = *Threshold* |
| *Step* | Relative shift from current position, expressed in 32-bit words:<br>if Step[7] = 0, then PC = PC + Step[6:0]<br>if Step[7] = 1, then PC = PC - Step[6:0] |

**Note:**

All jump addresses are expressed in 32-bit words.

**Description**

The instruction executes a jump to a relative address, if the condition is true. The condition is the result of comparing the R0 register value and the *Threshold* value.

#### 2.5.2.6 JUMPS – Jump to a Relative Address (Conditional upon Stage Count Register)

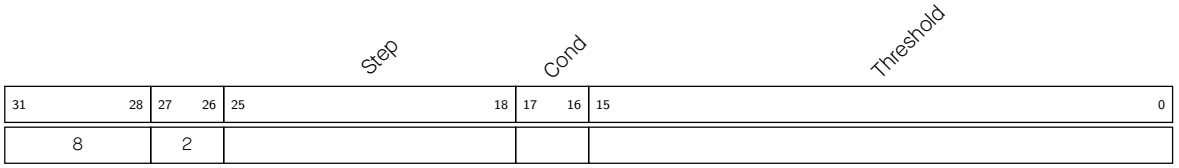



**Figure 2.5-13. Instruction Type - JUMPS**

| Operand | **Description** - see Figure 2.5-13 |
|---|---|
| *Threshold* | Threshold value for condition (see *Cond* below) to jump |
| *Cond* | Condition to jump:<br>1X - jump if *Stage_cnt* <= *Threshold*<br>00 - jump if *Stage_cnt* < *Threshold*<br>01 - jump if *Stage_cnt* >= *Threshold* |
| *Step* | Relative shift from current position, expressed in 32-bit words:<br>if Step[7] = 0, then PC = PC + Step[6:0]<br>if Step[7] = 1, then PC = PC - Step[6:0] |

**Note:**

- For more information about the stage count register, please refer to Section 2.5.2.1.
- All jump addresses are expresses in 32-bit words.

**Description**

The instruction executes a jump to a relative address if the condition is true. The condition itself is the result of comparing the value of *Stage_cnt* (stage count register) and the *Threshold* value.

#### 2.5.2.7 HALT – End the Program

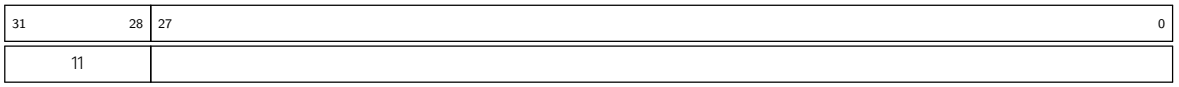


Figure 2.5-14. Instruction Type - HALT

**Description**

The instruction ends the operation of the ULP-FSM and puts it into power-down mode.

**Note:**

After executing this instruction, the ULP coprocessor wakeup timer gets started.

#### 2.5.2.8 WAKE – Wake up the Chip

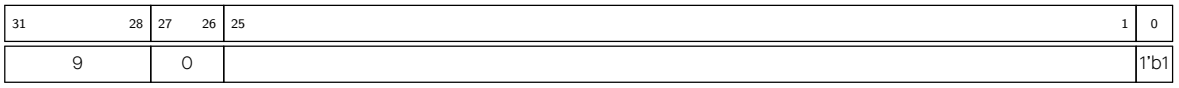



Figure 2.5-15. Instruction Type - WAKE

**Description**

This instruction sends an interrupt from the ULP-FSM to the RTC controller.

- If the chip is in Deep-sleep mode, and the ULP wakeup timer is enabled, the above-mentioned interrupt will wake up the chip.
- If the chip is not in Deep-sleep mode, and the ULP interrupt bit RTC_CNTL_ULP_CP_INT_ENA is set in register RTC_CNTL_INT_ENA_REG, an RTC interrupt will be triggered.

#### 2.5.2.9 WAIT – Wait for a Number of Cycles

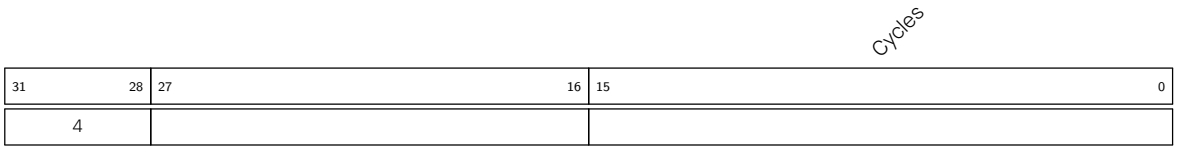



Figure 2.5-16. Instruction Type - WAIT

| Operand | **Description** - see Figure 2.5-16 |
|---|---|
| *Cycles* | The number of cycles to wait |

**Description**

The instruction will delay the ULP-FSM for a given number of cycles.

#### 2.5.2.10 TSENS – Take Measurement with Temperature Sensor

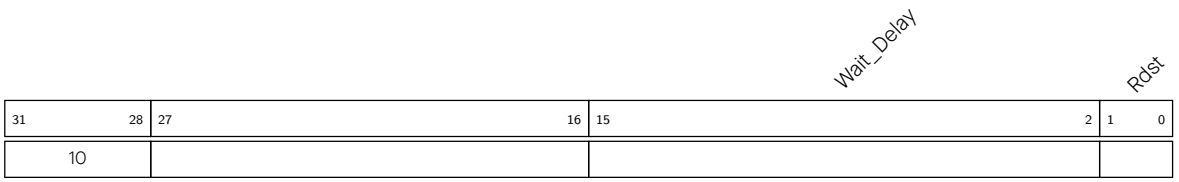



Figure 2.5-17. Instruction Type - TSENS

| Operand | **Description** - see Figure 2.5-17 |
|---|---|
| *Rdst* | Destination Register R[0-3], results will be stored in this register. |
| *Wait_Delay* | Number of cycles used to perform the measurement. |

**Description**

Increasing the measurement cycles *Wait_Delay* helps improve the accuracy and optimize the result. The instruction performs measurement via temperature sensor and stores the result into a general purpose register.

### 2.5.2.11 ADC – Take Measurement with ADC

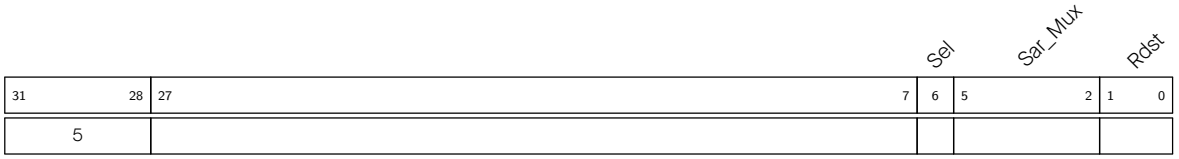



Figure 2.5-18. Instruction Type - ADC

| Operand | **Description** - see Figure 2.5-18 |
|---|---|
| *Rdst* | Destination Register R[0-3], results will be stored in this register. |
| *Sar_Mux* | Enable SAR ADC channel. Channel No. is [Sar_Mux - 1]. For more information, see Chapter 39 *On-Chip Sensors and Analog Signal Processing*. |
| *Sel* | Select ADC. 0: select SAR ADC1; 1: select SAR ADC2, see Table 2.5-6. |

Table 2.5-6. Input Signals Measured Using the ADC Instruction

| Pad/Signal/GPIO | *Sar_Mux* | ADC Selection (*Sel*) |
|---|---|---|
| GPIO1 | 1 | *Sel* = 0, select SAR ADC1 |
| GPIO2 | 2 | |
| GPIO3 | 3 | |
| GPIO4 | 4 | |
| GPIO5 | 5 | |
| GPIO6 | 6 | |
| GPIO7 | 7 | |
| GPIO8 | 8 | |
| GPIO9 | 9 | |
| GPIO10 | 10 | |
| GPIO11 | 1 | *Sel* = 1, select SAR ADC2 |
| GPIO12 | 2 | |
| GPIO13 | 3 | |
| GPIO14 | 4 | |
| XTAL_32k_P | 5 | |
| XTAL_32k_N | 6 | *Sel* = 1, select SAR ADC2 |
| GPIO17 | 7 | |
| GPIO18 | 8 | |
| GPIO19 | 9 | |
| GPIO20 | 10 | |

### 2.5.2.12 REG_RD – Read from Peripheral Register

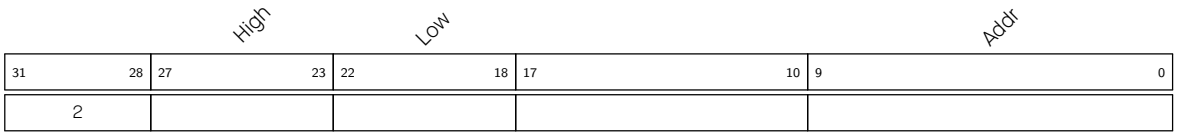



Figure 2.5-19. Instruction Type - REG_RD

| **Operand** | **Description** - see Figure 2.5-19 |
|---|---|
| *Addr* | Peripheral register address, in 32-bit words |
| *Low* | Register start bit number |
| *High* | Register end bit number |

**Description**

The instruction reads up to 16 bits from a peripheral register into a general-purpose register:

R0 = REG[Addr][High:Low]

In case of more than 16 bits being requested, i.e., *High* - *Low* + 1 > 16, then the instruction will return [Low+15:Low].

**Note:**

- This instruction can access registers in RTC_CNTL, RTC_IO, SENS, and RTC_I2C peripherals. Address of the register, as seen from the ULP coprocessor (addr_ulp), can be calculated from the address of the same register on the main bus (addr_bus), as follows:

  addr_ulp = (addr_bus - DR_REG_RTCCNTL_BASE)/4

- The *addr_ulp* is expressed in 32-bit words (not in bytes), and value 0 maps onto the DR_REG_RTCCNTL_BASE (as seen from the main CPU). Thus, 10 bits of address cover a 4096-byte range of peripheral register space, including regions DR_REG_RTCCNTL_BASE (0x6000800), DR_REG_RTCIO_BASE (0x60008400), DR_REG_SENS_BASE (0x60008800), and DR_REG_RTC_I2C_BASE (0x60008C00). For more information about address mapping, see Section 2.8.

### 2.5.2.13 REG_WR – Write to Peripheral Register

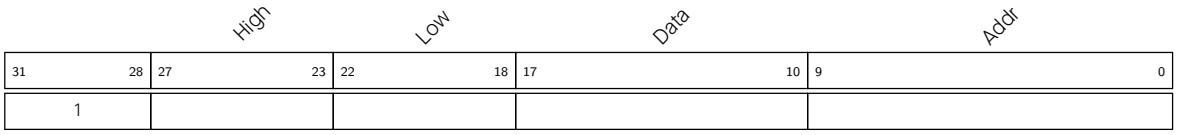



Figure 2.5-20. Instruction Type - REG_WR

| **Operand** | **Description** - see Figure 2.5-20 |
|---|---|
| *Addr* | Register address, expressed in 32-bit words |
| *Data* | Value to write, 8 bits |
| *Low* | Register start bit number |
| *High* | Register end bit number |

**Description**

This instruction writes up to 8 bits from an immediate data value into a peripheral register.

REG[Addr][High:Low] = Data

If more than 8 bits are requested, i.e., *High* - *Low* + 1 > 8, then the instruction will pad with zeros the bits above the eighth bit.

**Note:**

See notes regarding *addr_ulp* in Section 2.5.2.12.

# 2.6 ULP-RISC-V

## 2.6.1 Features

- Support RV32IMC instruction set
- Thirty-two 32-bit general-purpose registers
- 32-bit multiplier and divider
- Support for interrupts

## 2.6.2 Multiplier and Divider

ULP-RISC-V has an independent multiplication and division unit. The efficiency of multiplication and division instructions is shown in the following table.

**Table 2.6-1. Instruction Efficiency**

| Operation | Instruction | Execution Cycle | Instruction Description |
|---|---|---|---|
| Multiply | MUL | 34 | Multiply two 32-bit integers and return the lower 32-bit of the result |
| | MULH | 66 | Multiply two 32-bit signed integers and return the higher 32-bit of the result |
| | MULHU | 66 | Multiply two 32-bit unsigned integers and return the higher 32-bit of the result |
| | MULHSU | 66 | Multiply a 32-bit signed integer with a unsigned integer and return the higher 32-bit of the result |

| Operation | Instruction | Execution Cycle | Instruction Description |
|---|---|---|---|
| Divide | DIV | 34 | Divide a 32-bit integer by a 32-bit integer and return the quotient |
| | DIVU | 34 | Divide a 32-bit unsigned integer by a 32-bit unsigned integer and return the quotient |
| | REM | 34 | Divide a 32-bit signed integer by a 32-bit signed integer and return the remainder |
| | REMU | 34 | Divide a 32-bit unsigned integer by a 32-bit unsigned integer and return the remainder |

## 2.6.3 ULP-RISC-V Interrupts

### 2.6.3.1 Introduction

The interrupt controller of ULP-RISC-V is implemented by using a customized instruction set, instead of RISC-V Privileged ISA specification, aiming to reduce the size of ULP-RISC-V.

### 2.6.3.2 Interrupt Controller

ULP-RISC-V has 32 interrupt sources, but only four of them are available in the real design, as shown in the table below, including:

- internal sources: INT 0 ~ INT 2, triggered by internal interrupt events.
- external source: INT 31, triggered by the peripheral interrupts of ESP32-S3.

| Type | IRQ | Triggered by |
|---|---|---|
| Internal | 0 | Internal timer interrupt |
| Internal | 1 | EBREAK/ECALL or Illegal Instruction |
| Internal | 2 | BUS Error (Unaligned Memory Access) |
| External | 31 | RTC peripheral interrupts |

**Table 2.6-2. ULP-RISC-V Interrupt Sources**

**Note:**

If illegal instruction interrupt or bus error interrupt is disabled, ULP-RISC-V goes to HALT when the two errors occur.

ULP-RISC-V provides four 32-bit interrupt registers, Q0 ~ Q3, to handle interrupt service routine (ISR). Table 2.6-3 shows the function of each register.

| Register | Function |
|---|---|
| Q0 | Store the returned address. If the interrupt instruction is a compressed one, the lowest bit of this register will be set. |
| Q1 | Contain a bitmask of all IRQs to be handled. This means one call to the interrupt handler needs to service more than one IRQ when more than one bit is set in Q1. |
| Q2 | Reserved. An option for ISR to store data. |
| Q3 | Reserved. An option for ISR to store data. |

Table 2.6-3. ULP-RISC-V Interrupt Registers

**Note:**

- If more than one bits in Q1 are set, all the corresponding interrupts will call the same ISR. For such reason, users need to program ISR to check the interrupt number and execute corresponding program.
- After ULP-RISC-V is reset, all the interrupts are disabled.
- All ISR entries are located at 0x10, and the reset entry is at 0x0.

#### 2.6.3.3 Interrupt Handling

When an interrupt occurs, ULP-RISC-V performs the following operations:

1. Saves the current PC to Q0.
2. Saves the interrupt being responded to in Q1.
3. Jumps to the interrupt entry point (0x10).

After ULP-RISC-V enters the interrupt entry point, please save the context and use Q2 and Q3 as temporary storage registers for backup. Before it exits the interrupt, restore the context. Then, exit the interrupt by executing the retirq instruction and jump to Q0. ULP does not support interrupt nesting and is not interrupted by any other interrupt before retirq is executed.

#### 2.6.3.4 Interrupt Instructions

All these interrupt instructions are standard R-type instructions, with the same OpCode of custom0 (0001011). Figure 2.6-1 shows the format of standard R-type instructions. Note the fields funct3 (f3) and rs2 are ignored in these instructions.

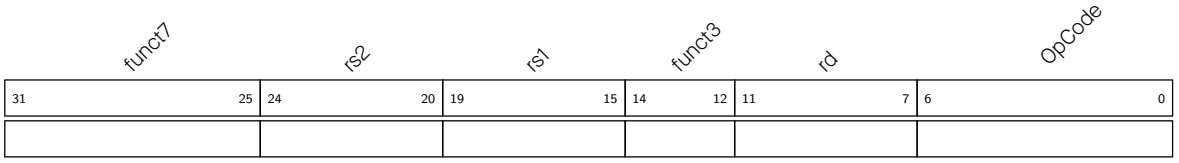



Figure 2.6-1. Standard R-type Instruction Format

**Instruction: getq rd,qs**

This instruction copies the value of Q*x* into a general purpose register rd.

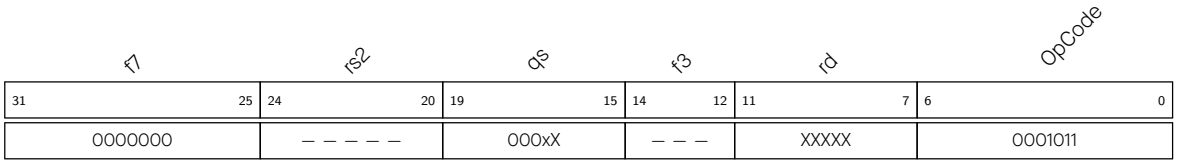



Figure 2.6-2. Interrupt Instruction - getq rd, qs

| Operand | Description - See Figure 2.6-2 |
|---|---|
| *rd* | Target general purpose register, holds the value of interrupt register specified by qs. |
| *qs* | Address of interrupt register Q*x*. |
| *f7* | Interrupt instruction number. |

### Instruction: setq qd,rs

This instruction copies the value of general purpose register rs to Q*x*.

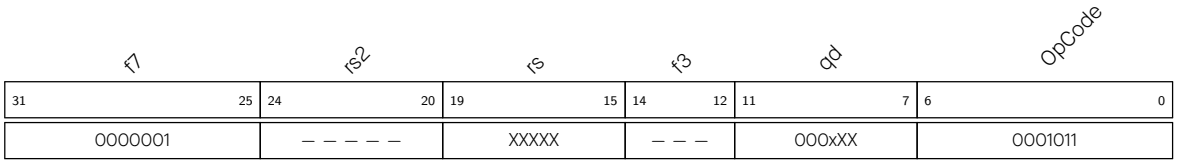



Figure 2.6-3. Interrupt Instruction - setq qd,rs

| Operand | Description - See Figure 2.6-3 |
|---|---|
| *qd* | Target interrupt register |
| *rs* | Source general purpose register, stores the value to be written to interrupt register. |
| *f7* | Interrupt instruction number. |

### Instruction: retirq

This instruction copies the value of Q*0* to CPU PC, and enables interrupt again.

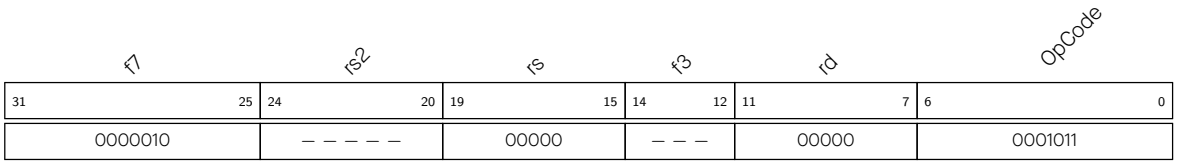



Figure 2.6-4. Interrupt Instruction - retirq

| Operand | Description - See Figure 2.6-4 |
|---|---|
| *f7* | Interrupt instruction number. |

### Instruction: maskirq rd,rs

This instruction copies the value of the register IRQ Mask to the register rd, and copies the value of register rs to IRQ Mask.

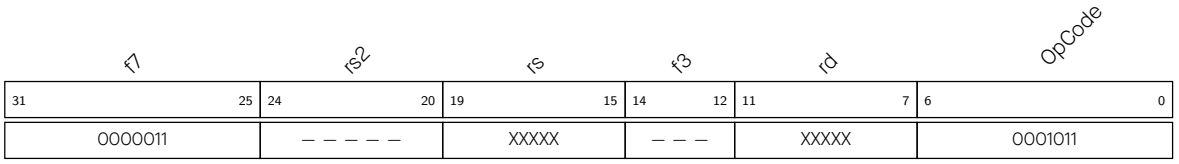


Figure 2.6-5. Interrupt Instruction — Maskirq rd rs

| Operand | Description - See Figure 2.6-5 |
|---|---|
| *rd* | Target general purpose register, stores current value of register Q1. |
| *rs* | Source general purpose register, stores the value to be written to Q1. |
| *f7* | Interrupt instruction number. |

#### 2.6.3.5 RTC Peripheral Interrupts

The interrupts from some sensors, software, and RTC I2C can be routed to ULP-RISC-V. To enable the interrupts, please set the register SENS_SAR_COCPU_INT_ENA_REG, see Table 2.6-4.

| Enable bit | Interrupt | Description |
|---|---|---|
| 0 | TOUCH_DONE_INT | Triggered when the touch sensor completes the scan of a channel |
| 1 | TOUCH_INACTIVE_INT | Triggered when the touch pad is released |
| 2 | TOUCH_ACTIVE_INT | Triggered when the touch pad is touched |
| 3 | SARADC1_DONE_INT | Triggered when SAR ADC1 completes the conversion one time |
| 4 | SARADC2_DONE_INT | Triggered when SAR ADC2 completes the conversion one time |
| 5 | TSENS_DONE_INT | Triggered when the temperature sensor completes the dump of its data |
| 6 | RISCV_START_INT | Triggered when ULP-RISC-V powers on and starts working |
| 7 | SW_INT | Triggered by software |
| 8 | SWD_INT | Triggered by timeout of Super Watchdog (SWD) |
| 9 | TOUCH_TIME_OUT_INT | Triggered by touch pad sampling timeout |
| 10 | TOUCH_APPROACH_LOOP_DONE_INT | Triggered when touch pad completes an APPROACH sampling |
| 11 | TOUCH_SCAN_DONE_INT | Triggered when touch pad completes the scan of the final channel |

Table 2.6-4. ULP-RISC-V Interrupt List

**Note:**

- Besides the above-mentioned interrupts, ULP-RISC-V can also handle the interrupt from RTC_IO by simply configuring RTC_IO as input mode. Users can configure RTCIO_GPIO_PIN*n*_INT_TYPE to select the interrupt trigger modes. For more details about RTC_IO configuration, see Chapter 6 *IO MUX and GPIO Matrix (GPIO, IO MUX)*.
- The interrupt from RTC_IO can be cleared by releasing RTC_IO and its source can be read from the register RTCIO_RTC_GPIO_STATUS_REG.
- The SW_INT interrupt is generated by configuring the register RTC_CNTL_COCPU_SW_INT_TRIGGER.
- For the information about RTC I2C interrupts, please refer to Section 2.7.4.

## 2.7 RTC I2C Controller

ULP coprocessor reads from or writes to external I2C slave devices via RTC I2C controller.

### 2.7.1 Connecting RTC I2C Signals

SDA and SCL signals can be mapped onto two out of the four GPIO pins, which are identified in Table RTC_MUX Pad List in Chapter 6 *IO MUX and GPIO Matrix (GPIO, IO MUX)*, using the register RTCIO_SAR_I2C_IO_REG.

### 2.7.2 Configuring RTC I2C

Before ULP coprocessor can communicate using I2C instruction, RTC I2C need to be configured. Configuration is performed by writing certain timing parameters into the RTC I2C registers. This can be done by the program running on the main CPU, or by the ULP coprocessor itself.

**Note:**
The timing parameters are configured in cycles of RTC_FAST_CLK running at 17.5 MHz.

1. Set the low and high SCL half-periods by configuring RTC_I2C_SCL_LOW_PERIOD_REG and RTC_I2C_SCL_HIGH_PERIOD_REG in RTC_FAST_CLK cycles (e.g., RTC_I2C_SCL_LOW_PERIOD_REG = 40, RTC_I2C_SCL_HIGH_PERIOD_REG = 40 for 100 kHz frequency).
2. Set the number of cycles between the SDA switch and the falling edge of SCL by using RTC_I2C_SDA_DUTY_REG in RTC_FAST_CLK (e.g., RTC_I2C_SDA_DUTY_REG = 16).
3. Set the waiting time after the START signal by using RTC_I2C_SCL_START_PERIOD_REG (e.g., RTC_I2C_SCL_START_PERIOD = 30).
4. Set the waiting time before the END signal by using RTC_I2C_SCL_STOP_PERIOD_REG (e.g., RTC_I2C_SCL_STOP_PERIOD = 44).
5. Set the transaction timeout by using RTC_I2C_TIME_OUT_REG (e.g., RTC_I2C_TIME_OUT_REG = 200).
6. Configure the RTC I2C controller into master mode by setting the bit RTC_I2C_MS_MODE in RTC_I2C_CTRL_REG.
7. Configure the address(es) of external slave(s):
   - If ULP-RISC-V or main CPU is used, then write the slave address to SENS_SAR_I2C_SLAVE_ADDR.
   - If ULP-FSM is used, then write the slave address to SENS_I2C_SLAVE_ADDR*n* (*n*: 0-7)

   Up to eight slave addresses can be pre-programmed. One of these addresses can then be selected for each transaction as part of the RTC I2C instruction.

Once RTC I2C is configured, the main CPU or the ULP coprocessor can communicate with the external I2C devices.

### 2.7.3 Using RTC I2C

#### 2.7.3.1 Instruction Format

The format of RTC I2C instruction is basically consistent with that of I2C0/I2C1, see Section I2C CMD Controller in Chapter *27 I2C Controller (I2C)* except the following:

- RTC I2C has different op_code mapping:
  - RSTART: op_code = 0
  - WRITE: op_code = 1

  - READ: op_code = 2
  - STOP: op_code = 3
  - END: op_code = 4
- RTC I2C provides fixed instructions for different operations, as follows:
  - Command 0 ~ Command 1: specifically for I2C write operation
  - Command 2 ~ Command 6: specifically for I2C read operation

**Note:** All slave addresses are expressed in 7 bits.

#### 2.7.3.2 I2C_RD - I2C Read Workflow

Preparation for RTC I2C read:

- Configure the instruction list of RTC I2C (see Section CMD_Controller in Chapter *27 I2C Controller (I2C)*), including instruction order, instruction code, read data number (byte_num), and other information.
- Configure the slave register address by setting the register SENS_SAR_I2C_REG_ADDR.
- Start RTC I2C transmission by setting SENS_SAR_I2C_START_FORCE and SENS_SAR_I2C_START.
- When an RTC_I2C_RX_DATA_INT interrupt is received, transfer the read data stored in RTC_I2C_RDATA to SRAM RTC slow memory, or use the data directly.

The I2C_RD instruction performs the following operations (see Figure 2.7-1):

1. Master generates a START signal.
2. Master sends slave address, with r/w bit set to 0 ("write"). Slave address is obtained from SENS_I2C_SLAVE_ADDR*n*.
3. Slave generates ACK.
4. Master sends slave register address.
5. Slave generates ACK.
6. Master generates a repeated START (RSTART) signal.
7. Master sends slave address, with r/w bit set to 1 ("read").
8. Slave sends one byte of data.
9. Master checks whether the number of transmitted bytes reaches the number set by the current instruction (byte_num). If yes, master jumps out of the read instruction and sends an NACK signal. Otherwise master repeats Step 8 and waits for the slave to send the next byte.
10. Master generates a STOP signal and stops reading.

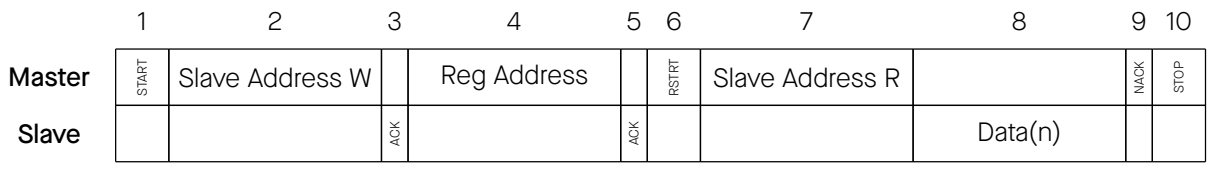



**Figure 2.7-1. I2C Read Operation**

**Note:**
The RTC I2C peripheral samples the SDA signals on the falling edge of SCL. If the slave changes SDA in less than 0.38 ms, the master may receive incorrect data.

#### 2.7.3.3 I2C_WR - I2C Write Workflow

Preparation for RTC I2C write:

- Configure RTC I2C instruction list, including instruction order, instruction code, and the data to be written in byte (byte_num). See the configuration of I2C0/I2C1 in Section CMD_Controller in Chapter *27 I2C Controller (I2C)*.
- Configure the slave register address by setting the register SENS_SAR_I2C_REG_ADDR, and the data to be transmitted in SENS_SAR_I2C_WDATA.
- Set SENS_SAR_I2C_START_FORCE and SENS_SAR_I2C_START to start the transmission.
- Update the next data to be transmitted in SENS_SAR_I2C_WDATA, each time when an RTC_I2C_TX_DATA_INT interrupt is received.

The I2C_WR instruction performs the following operations, see Figure 2.7-2.

1. Master generates a START signal.
2. Master sends slave address, with r/w bit set to 0 ("write"). Slave address is obtained from SENS_I2C_SLAVE_ADDR*n*.
3. Slave generates ACK.
4. Master sends slave register address.
5. Slave generates ACK.
6. Master generates a repeated START (RSTART) signal.
7. Master sends slave address, with r/w bit set to 0 ("write").
8. Master sends one byte of data.
9. Slave generates ACK. Master checks whether the number of transmitted bytes reaches the number set by the current instruction (byte_num). If yes, master jumps out of the write instruction and starts the next instruction. Otherwise the master repeats Step 8 and sends the next byte.
10. Master generates a STOP signal and stops the transmission.

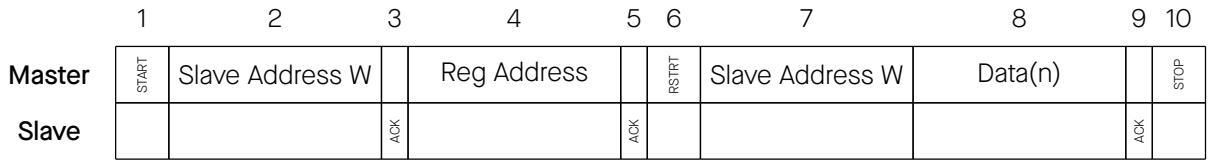



Figure 2.7-2. I2C Write Operation

#### 2.7.3.4 Detecting Error Conditions

Applications can query specific bits in the RTC_I2C_INT_ST_REG register to check if the transaction is successful. To enable checking for specific communication events, their corresponding bits should be set in

register RTC_I2C_INT_ENA_REG. Note that the bit map is shifted by 1. If a specific communication event is detected and its corresponding bit in register RTC_I2C_INT_ST_REG is set, the event can then be cleared using register RTC_I2C_INT_CLR_REG.

### 2.7.4 RTC I2C Interrupts

- RTC_I2C_SLAVE_TRAN_COMP_INT: Triggered when the slave finishes the transaction.
- RTC_I2C_ARBITRATION_LOST_INT: Triggered when the master loses control of the bus.
- RTC_I2C_MASTER_TRAN_COMP_INT: Triggered when the master completes the transaction.
- RTC_I2C_TRANS_COMPLETE_INT: Triggered when a STOP signal is detected.
- RTC_I2C_TIME_OUT_INT: Triggered by time out event.
- RTC_I2C_ACK_ERR_INT: Triggered by ACK error.
- RTC_I2C_RX_DATA_INT: Triggered when data is received.
- RTC_I2C_TX_DATA_INT: Triggered when data is transmitted.
- RTC_I2C_DETECT_START_INT: Triggered when a START signal is detected.

## 2.8 Address Mapping

Table 2.8-1 shows the address mapping and available base registers for the peripherals accessible by ULP coprocessors.

Table 2.8-1. Address Mapping

| Peripheral(s) | Base Register | Main Bus Address | ULP-FSM Base | ULP-RISC-V Base |
|---|---|---|---|---|
| RTC Control | DR_REG_RTCCNTL_BASE | 0x60008000 | 0x8000 | 0x8000 |
| RTC GPIO | DR_REG_RTC_IO_BASE | 0x60008400 | 0x8400 | 0xA400 |
| ADC, Touch, TSENS | DR_REG_SENS_BASE | 0x60008800 | 0x8800 | 0xC800 |
| RTC I2C | DR_REG_RTC_I2C_BASE | 0x60008C00 | 0x8C00 | 0xEC00 |

To find more information about registers for these peripherals, please check the following chapters.

Table 2.8-2. Description of Registers for Peripherals Accessible by ULP Coprocessors

| Registers Available for Peripherals | Described in Which Chapter |
|---|---|
| Registers for RTC Control | Chapter 10 *Low-power Management (RTC_CNTL)* |
| Registers for RTC GPIO | Chapter 6 *IO MUX and GPIO Matrix (GPIO, IO MUX)* |
| Registers for ARC, Touch, TSENS | Chapter 39 *On-Chip Sensors and Analog Signal Processing* |
| Registers for RTC I2C | Section 2.9 *Register Summary* in this chapter |

## 2.9 Register Summary

The following registers are used in ULP coprocessor:

- ULP (ALWAYS_ON) registers: not reset due to power down of RTC_PERI domain. See Chapter 10 *Low-power Management (RTC_CNTL)*.
- ULP (RTC_PERI) registers: reset due to power down of RTC_PERI domain. See Chapter 10 *Low-power Management (RTC_CNTL)*.
- RTC I2C registers: I2C related registers, including RTC I2C (RTC_PERI) and RTC I2C (I2C) registers.

### 2.9.1 ULP (ALWAYS_ON) Register Summary

The addresses in this section are relative to low-power management base address provided in Table 4.3-3 in Chapter 4 *System and Memory*.

| Name | Description | Address | Access |
|---|---|---|---|
| **ULP Timer Registers** | | | |
| RTC_CNTL_ULP_CP_TIMER_REG | Configure the timer | 0x00FC | varies |
| RTC_CNTL_ULP_CP_TIMER_1_REG | Configure sleep cycle of the timer | 0x0134 | R/W |
| **ULP-FSM Register** | | | |
| RTC_CNTL_ULP_CP_CTRL_REG | ULP-FSM configuration register | 0x0100 | R/W |
| **ULP-RISC-V Register** | | | |
| RTC_CNTL_COCPU_CTRL_REG | ULP-RISC-V configuration register | 0x0104 | varies |

### 2.9.2 ULP (RTC_PERI) Register Summary

The addresses in this section are relative to low-power management base address provided in Table 4.3-3 in Chapter 4 *System and Memory*.

| Name | Description | Address | Access |
|---|---|---|---|
| **ULP-RISC-V Registers** | | | |
| SENS_ SAR_COCPU_INT_RAW_REG | Interrupt raw bit of ULP-RISC-V | 0x00E8 | RO |
| SENS_SAR_COCPU_INT_ENA_REG | Interrupt enable bit of ULP-RISC-V | 0x00EC | R/W |
| SENS_SAR_COCPU_INT_ST_REG | Interrupt status bit of ULP-RISC-V | 0x00F0 | RO |
| SENS_SAR_COCPU_INT_CLR_REG | Interrupt clear bit of ULP-RISC-V | 0x00F4 | WO |

### 2.9.3 RTC I2C (RTC_PERI) Register Summary

The addresses in this section are relative to low-power management base address + 0x0800 provided in Table 4.3-3 in Chapter 4 *System and Memory*.

| Name | Description | Address | Access |
|---|---|---|---|
| **RTC I2C Controller Register** | | | |
| SENS_SAR_I2C_CTRL_REG | Configure RTC I2C transmission | 0x0058 | R/W |
| **RTC I2C Slave Address Registers** | | | |
| SENS_SAR_SLAVE_ADDR1_REG | Configure slave addresses 0-1 of RTC I2C | 0x0040 | R/W |
| SENS_SAR_SLAVE_ADDR2_REG | Configure slave addresses 2-3 of RTC I2C | 0x0044 | R/W |
| SENS_SAR_SLAVE_ADDR3_REG | Configure slave addresses 4-5 of RTC I2C | 0x0048 | R/W |
| SENS_SAR_SLAVE_ADDR4_REG | Configure slave addresses 6-7 of RTC I2C | 0x004C | R/W |

### 2.9.4 RTC I2C (I2C) Register Summary

The addresses in this section are relative to low-power management base address + 0x0C00 provided in Table 4.3-3 in Chapter 4 *System and Memory*.

| Name | Description | Address | Access |
|---|---|---|---|
| **RTC I2C Signal Setting Registers** | | | |
| RTC_I2C_SCL_LOW_REG | Configure the low level width of SCL | 0x0000 | R/W |
| RTC_I2C_SCL_HIGH_REG | Configure the high level width of SCL | 0x0014 | R/W |
| RTC_I2C_SDA_DUTY_REG | Configure the SDA hold time after a negative SCL edge | 0x0018 | R/W |
| RTC_I2C_SCL_START_PERIOD_REG | Configure the delay between the SDA and SCL negative edge for a start condition | 0x001C | R/W |
| RTC_I2C_SCL_STOP_PERIOD_REG | Configure the delay between SDA and SCL positive edge for a stop condition | 0x0020 | R/W |
| **RTC I2C Control Registers** | | | |
| RTC_I2C_CTRL_REG | Transmission setting | 0x0004 | R/W |
| RTC_I2C_STATUS_REG | RTC I2C status | 0x0008 | RO |
| RTC_I2C_TO_REG | Configure RTC I2C timeout | 0x000C | R/W |
| RTC_I2C_SLAVE_ADDR_REG | Configure slave address | 0x0010 | R/W |
| **RTC I2C Interrupt Registers** | | | |
| RTC_I2C_INT_CLR_REG | Clear RTC I2C interrupt | 0x0024 | WO |
| RTC_I2C_INT_RAW_REG | RTC I2C raw interrupt | 0x0028 | RO |
| RTC_I2C_INT_ST_REG | RTC I2C interrupt status | 0x002C | RO |
| RTC_I2C_INT_ENA_REG | Enable RTC I2C interrupt | 0x0030 | R/W |
| **RTC I2C Status Register** | | | |
| RTC_I2C_DATA_REG | RTC I2C read data | 0x0034 | varies |
| **RTC I2C Command Registers** | | | |
| RTC_I2C_CMD0_REG | RTC I2C Command 0 | 0x0038 | varies |
| RTC_I2C_CMD1_REG | RTC I2C Command 1 | 0x003C | varies |
| RTC_I2C_CMD2_REG | RTC I2C Command 2 | 0x0040 | varies |
| RTC_I2C_CMD3_REG | RTC I2C Command 3 | 0x0044 | varies |
| RTC_I2C_CMD4_REG | RTC I2C Command 4 | 0x0048 | varies |
| RTC_I2C_CMD5_REG | RTC I2C Command 5 | 0x004C | varies |
| RTC_I2C_CMD6_REG | RTC I2C Command 6 | 0x0050 | varies |
| RTC_I2C_CMD7_REG | RTC I2C Command 7 | 0x0054 | varies |
| RTC_I2C_CMD8_REG | RTC I2C Command 8 | 0x0058 | varies |
| RTC_I2C_CMD9_REG | RTC I2C Command 9 | 0x005C | varies |
| RTC_I2C_CMD10_REG | RTC I2C Command 10 | 0x0060 | varies |
| RTC_I2C_CMD11_REG | RTC I2C Command 11 | 0x0064 | varies |
| RTC_I2C_CMD12_REG | RTC I2C Command 12 | 0x0068 | varies |
| RTC_I2C_CMD13_REG | RTC I2C Command 13 | 0x006C | varies |
| RTC_I2C_CMD14_REG | RTC I2C Command 14 | 0x0070 | varies |
| RTC_I2C_CMD15_REG | RTC I2C Command 15 | 0x0074 | varies |
| **Version register** | | | |

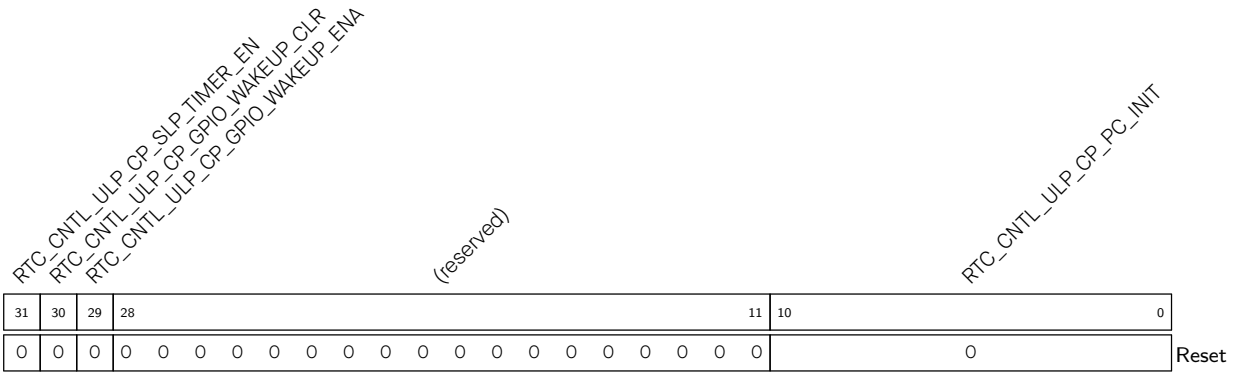
| Name | Description | Address | Access |
|---|---|---|---|
| RTC_I2C_DATE_REG | Version control register | 0x00FC | R/W |

# 2.10 Registers

## 2.10.1 ULP (ALWAYS_ON) Registers

The addresses in this section are relative to low-power management base address provided in Table 4.3-3 in Chapter 4 *System and Memory*.

**Register 2.1. RTC_CNTL_ULP_CP_TIMER_REG (0x00FC)**



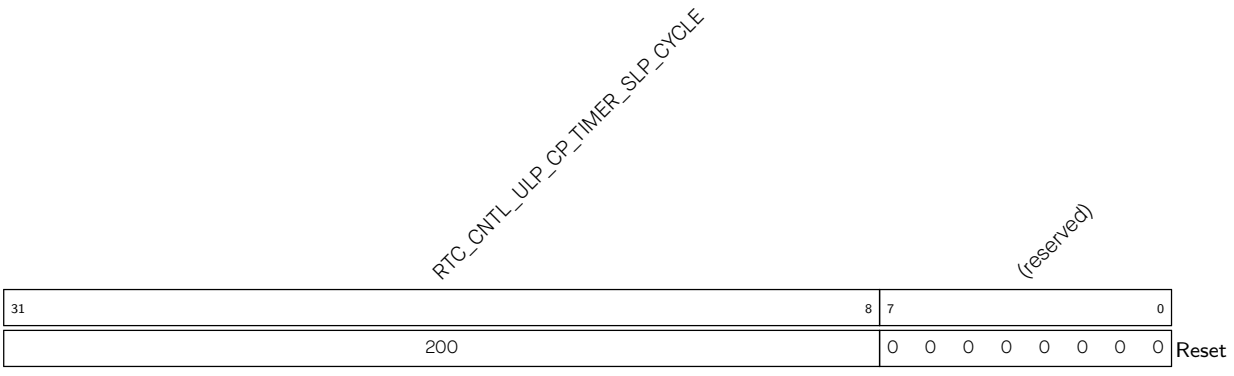

**RTC_CNTL_ULP_CP_PC_INIT** ULP coprocessor PC initial address. (R/W)

**RTC_CNTL_ULP_CP_GPIO_WAKEUP_ENA** Enable the option of ULP timer woken up by RTC GPIO. (R/W)

**RTC_CNTL_ULP_CP_GPIO_WAKEUP_CLR** Disable the option of ULP timer woken up by RTC GPIO. (WO)

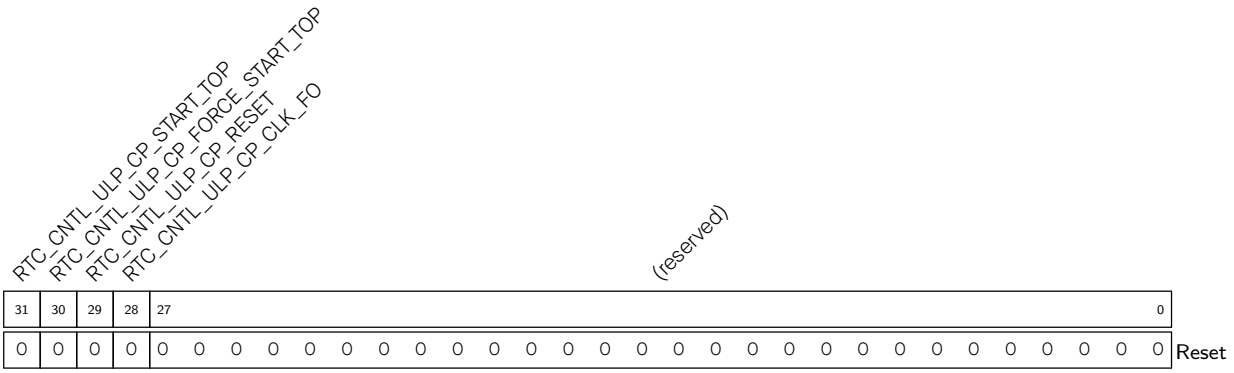
**RTC_CNTL_ULP_CP_SLP_TIMER_EN** ULP coprocessor timer enable bit. 0: Disable hardware timer; 1: Enable hardware timer. (R/W)

**Register 2.2. RTC_CNTL_ULP_CP_TIMER_1_REG (0x0134)**




**RTC_CNTL_ULP_CP_TIMER_SLP_CYCLE** Set sleep cycles for ULP coprocessor timer. (R/W)

Register 2.3. RTC_CNTL_ULP_CP_CTRL_REG (0x0100)



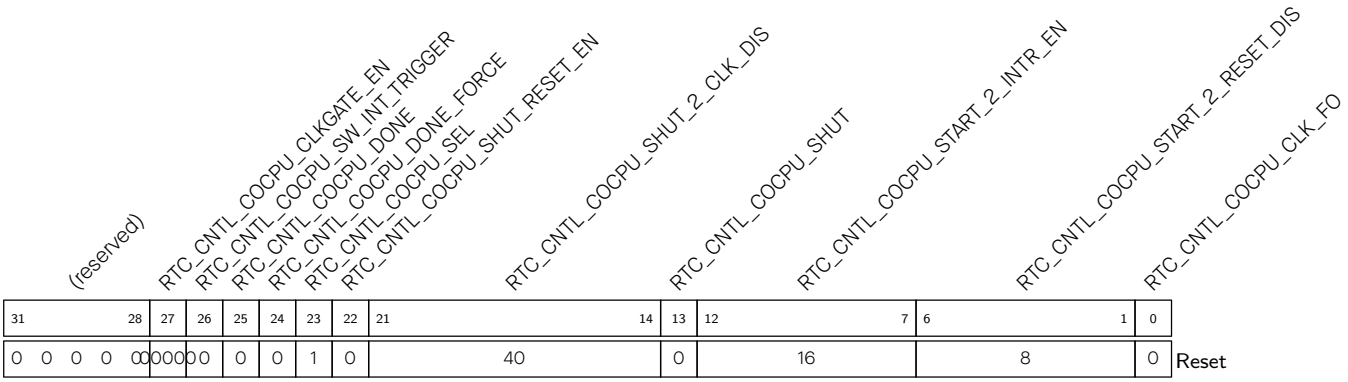

**RTC_CNTL_ULP_CP_CLK_FO** ULP-FSM clock force on. (R/W)

**RTC_CNTL_ULP_CP_RESET** ULP-FSM clock software reset. (R/W)

**RTC_CNTL_ULP_CP_FORCE_START_TOP** Write 1 to start ULP-FSM by software. (R/W)

**RTC_CNTL_ULP_CP_START_TOP** Write 1 to start ULP-FSM. (R/W)

**Register 2.4. RTC_CNTL_COCPU_CTRL_REG (0x0104)**




**RTC_CNTL_COCPU_CLK_FO** ULP-RISC-V clock force on. (R/W)

**RTC_CNTL_COCPU_START_2_RESET_DIS** Time from ULP-RISC-V startup to pull down reset. (R/W)

**RTC_CNTL_COCPU_START_2_INTR_EN** Time from ULP-RISC-V startup to send out RISCV_START_INT interrupt. (R/W)

**RTC_CNTL_COCPU_SHUT** Shut down ULP-RISC-V. (R/W)

**RTC_CNTL_COCPU_SHUT_2_CLK_DIS** Time from shut down ULP-RISC-V to disable clock. (R/W)

**RTC_CNTL_COCPU_SHUT_RESET_EN** This bit is used to reset ULP-RISC-V. (R/W)

**RTC_CNTL_COCPU_SEL** 0: select ULP-RISC-V; 1: select ULP-FSM. (R/W)

**RTC_CNTL_COCPU_DONE_FORCE** 0: select ULP-FSM DONE signal; 1: select ULP-RISC-V DONE signal. (R/W)

**RTC_CNTL_COCPU_DONE** DONE signal. Write 1 to this bit, ULP-RISC-V will go to HALT and the timer starts counting. (R/W)

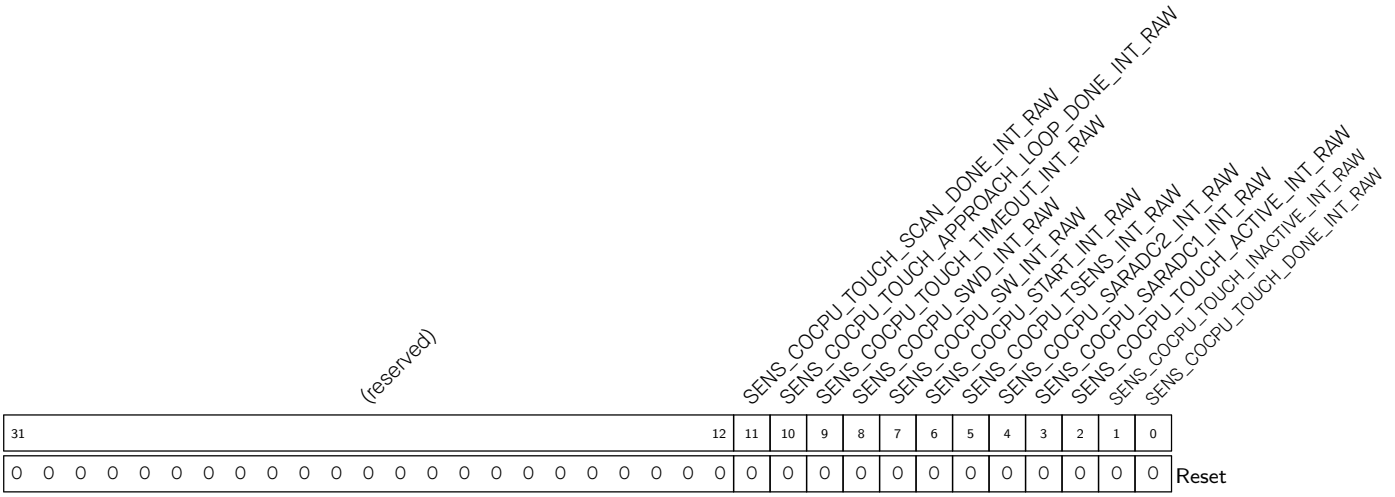
**RTC_CNTL_COCPU_SW_INT_TRIGGER** Trigger ULP-RISC-V register interrupt. (WO)

**RTC_CNTL_COCPU_CLKGATE_EN** Enable ULP-RICS-V clock gate. (WO)

## 2.10.2 ULP (RTC_PERI) Registers

The addresses in this section are relative to low-power management base address provided in Table 4.3-3 in Chapter 4 *System and Memory*.

Register 2.5. SENS_ SAR_COCPU_INT_RAW_REG (0x00E8)




**SENS_COCPU_TOUCH_DONE_INT_RAW** TOUCH_DONE_INT interrupt raw bit. (RO)

**SENS_COCPU_TOUCH_INACTIVE_INT_RAW** TOUCH_INACTIVE_INT interrupt raw bit. (RO)

**SENS_COCPU_TOUCH_ACTIVE_INT_RAW** TOUCH_ACTIVE_INT interrupt raw bit. (RO)

**SENS_COCPU_SARADC1_INT_RAW** SARADC1_DONE_INT interrupt raw bit. (RO)

**SENS_COCPU_SARADC2_INT_RAW** SARADC2_DONE_INT interrupt raw bit. (RO)

**SENS_COCPU_TSENS_INT_RAW** TSENS_DONE_INT interrupt raw bit. (RO)

**SENS_COCPU_START_INT_RAW** RISCV_START_INT interrupt raw bit. (RO)

**SENS_COCPU_SW_INT_RAW** SW_INT interrupt raw bit. (RO)

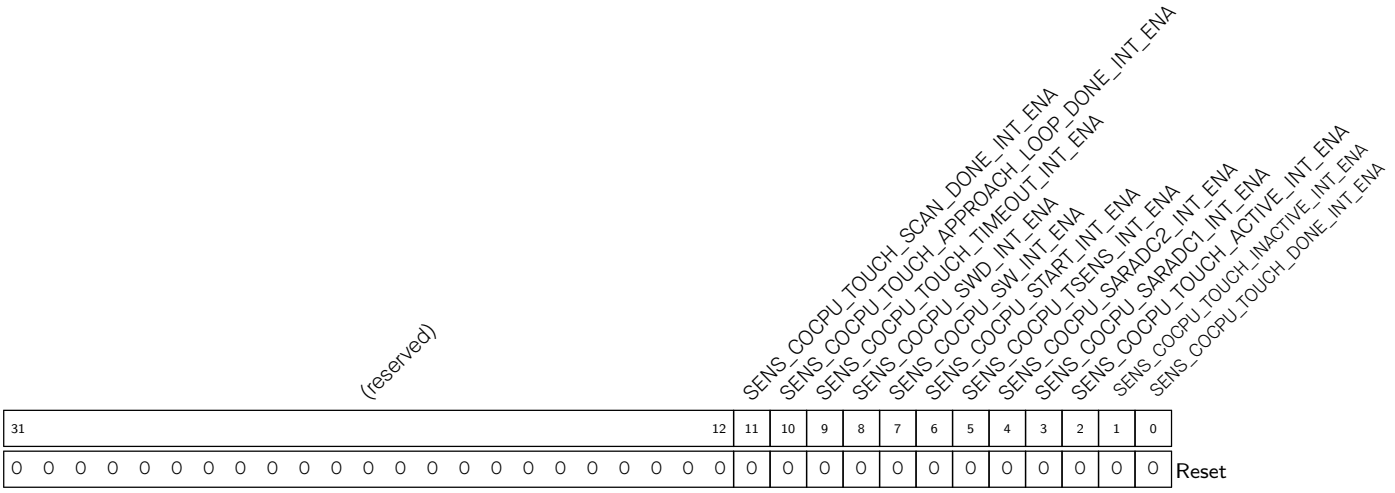
**SENS_COCPU_SWD_INT_RAW** SWD_INT interrupt raw bit. (RO)

**SENS_COCPU_TOUCH_TIMEOUT_INT_RAW** TOUCH_TIME_OUT interrupt raw bit. (RO)

**SENS_COCPU_TOUCH_APPROACH_LOOP_DONE_INT_RAW** TOUCH_APPROACH_LOOP_DONE_INT interrupt raw bit. (RO)

**SENS_COCPU_TOUCH_SCAN_DONE_INT_RAW** TOUCH_SCAN_DONE_INT interrupt raw bit. (RO)

Register 2.6. SENS_SAR_COCPU_INT_ENA_REG (0x00EC)




**SENS_COCPU_TOUCH_DONE_INT_ENA** TOUCH_DONE_INT interrupt enable bit. (R/W)

**SENS_COCPU_TOUCH_INACTIVE_INT_ENA** TOUCH_INACTIVE_INT interrupt enable bit. (R/W)

**SENS_COCPU_TOUCH_ACTIVE_INT_ENA** TOUCH_ACTIVE_INT interrupt enable bit. (R/W)

**SENS_COCPU_SARADC1_INT_ENA** SARADC1_DONE_INT interrupt enable bit. (R/W)

**SENS_COCPU_SARADC2_INT_ENA** SARADC2_DONE_INT interrupt enable bit. (R/W)

**SENS_COCPU_TSENS_INT_ENA** TSENS_DONE_INT interrupt enable bit. (R/W)

**SENS_COCPU_START_INT_ENA** RISCV_START_INT interrupt enable bit. (R/W)

**SENS_COCPU_SW_INT_ENA** SW_INT interrupt enable bit. (R/W)

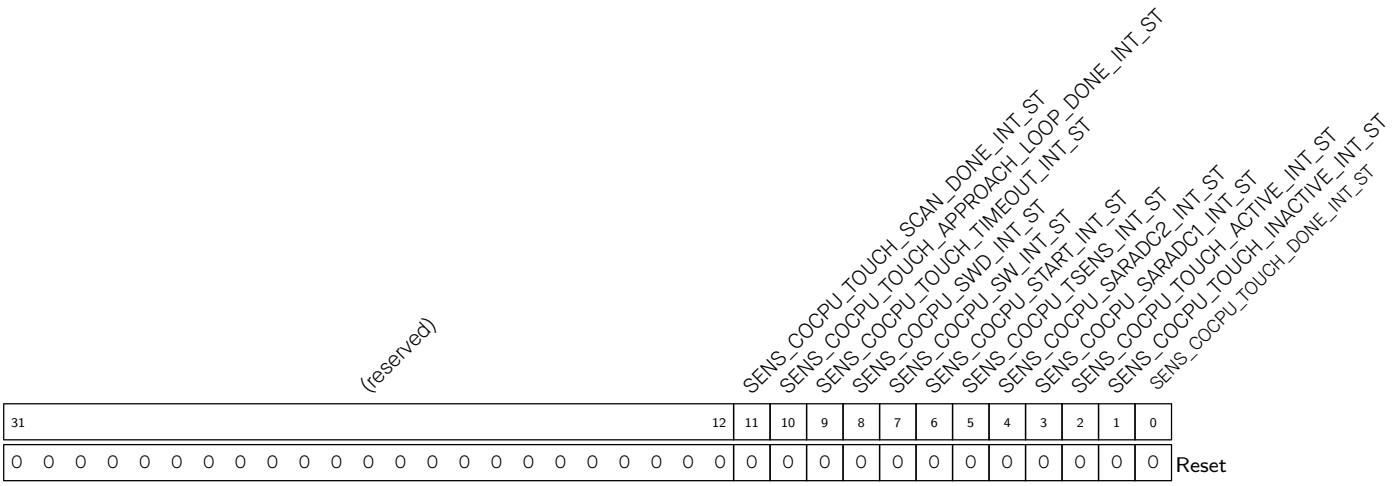
**SENS_COCPU_SWD_INT_ENA** SWD_INT interrupt enable bit. (R/W)

**SENS_COCPU_TOUCH_TIMEOUT_INT_ENA** TOUCH_TIME_OUT interrupt enable bit. (R/W)

**SENS_COCPU_TOUCH_APPROACH_LOOP_DONE_INT_ENA** TOUCH_APPROACH_LOOP_DONE_INT interrupt enable bit. (R/W)

**SENS_COCPU_TOUCH_SCAN_DONE_INT_ENA** TOUCH_SCAN_DONE_INT interrupt enable bit. (R/W)

Register 2.7. SENS_SAR_COCPU_INT_ST_REG (0x00F0)




**SENS_COCPU_TOUCH_DONE_INT_ST** TOUCH_DONE_INT interrupt status bit. (RO)

**SENS_COCPU_TOUCH_INACTIVE_INT_ST** TOUCH_INACTIVE_INT interrupt status bit. (RO)

**SENS_COCPU_TOUCH_ACTIVE_INT_ST** TOUCH_ACTIVE_INT interrupt status bit. (RO)

**SENS_COCPU_SARADC1_INT_ST** SARADC1_DONE_INT interrupt status bit. (RO)

**SENS_COCPU_SARADC2_INT_ST** SARADC2_DONE_INT interrupt status bit. (RO)

**SENS_COCPU_TSENS_INT_ST** TSENS_DONE_INT interrupt status bit. (RO)

**SENS_COCPU_START_INT_ST** RISCV_START_INT interrupt status bit. (RO)

**SENS_COCPU_SW_INT_ST** SW_INT interrupt status bit. (RO)

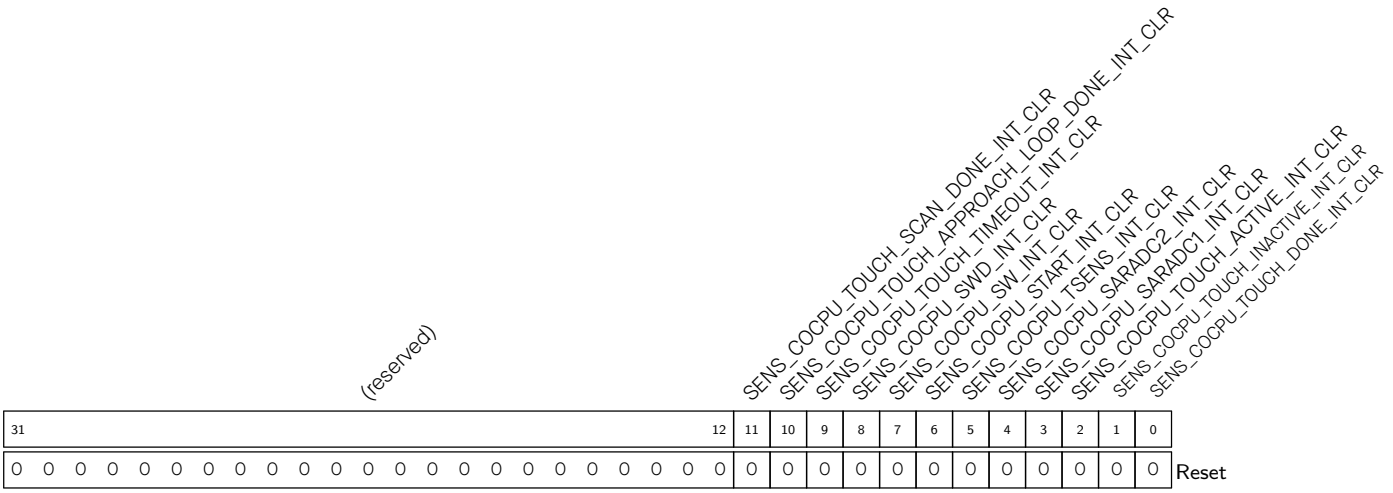
**SENS_COCPU_SWD_INT_ST** SWD_INT interrupt status bit. (RO)

**SENS_COCPU_TOUCH_TIMEOUT_INT_ST** TOUCH_TIME_OUT interrupt status bit. (RO)

**SENS_COCPU_TOUCH_APPROACH_LOOP_DONE_INT_ST** TOUCH_APPROACH_LOOP_DONE_INT interrupt status bit. (RO)

**SENS_COCPU_TOUCH_SCAN_DONE_INT_ST** TOUCH_SCAN_DONE_INT interrupt status bit. (RO)

Register 2.8. SENS_SAR_COCPU_INT_CLR_REG (0x00F4)




**SENS_COCPU_TOUCH_DONE_INT_CLR** TOUCH_DONE_INT interrupt clear bit. (WO)

**SENS_COCPU_TOUCH_INACTIVE_INT_CLR** TOUCH_INACTIVE_INT interrupt clear bit. (WO)

**SENS_COCPU_TOUCH_ACTIVE_INT_CLR** TOUCH_ACTIVE_INT interrupt clear bit. (WO)

**SENS_COCPU_SARADC1_INT_CLR** SARADC1_DONE_INT interrupt clear bit. (WO)

**SENS_COCPU_SARADC2_INT_CLR** SARADC2_DONE_INT interrupt clear bit. (WO)

**SENS_COCPU_TSENS_INT_CLR** TSENS_DONE_INT interrupt clear bit. (WO)

**SENS_COCPU_START_INT_CLR** RISCV_START_INT interrupt clear bit. (WO)

**SENS_COCPU_SW_INT_CLR** SW_INT interrupt clear bit. (WO)

**SENS_COCPU_SWD_INT_CLR** SWD_INT interrupt clear bit. (WO)

**SENS_COCPU_TOUCH_TIMEOUT_INT_CLR** TOUCH_TIME_OUT interrupt clear bit. (WO)

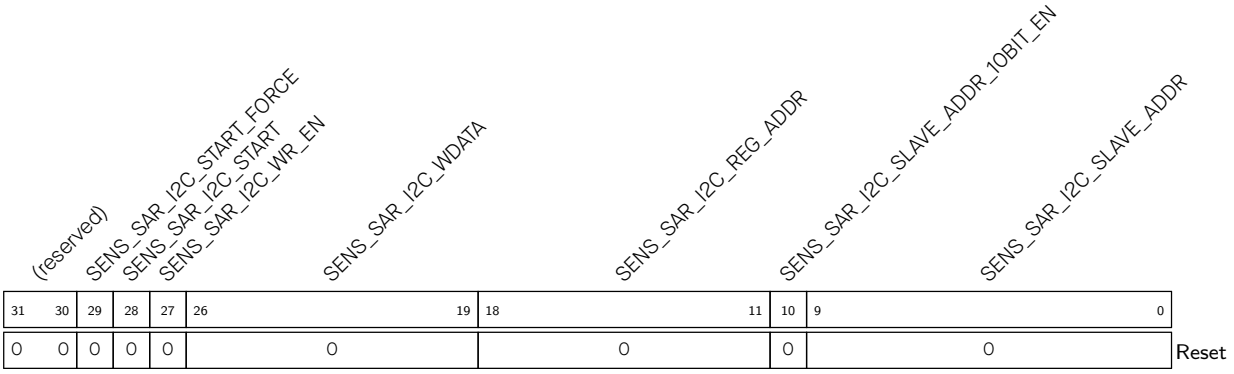
**SENS_COCPU_TOUCH_APPROACH_LOOP_DONE_INT_CLR** TOUCH_APPROACH_LOOP_DONE_INT interrupt clear bit. (WO)

**SENS_COCPU_TOUCH_SCAN_DONE_INT_CLR** TOUCH_SCAN_DONE_INT interrupt clear bit. (WO)

### 2.10.3 RTC I2C (RTC_PERI) Registers

The addresses in this section are relative to low-power management base address + 0x0800 provided in Table 4.3-3 in Chapter 4 *System and Memory*.

Register 2.9. SENS_SAR_I2C_CTRL_REG (0x0058)




**SENS_SAR_I2C_SLAVE_ADDR** Configures the slave address. (R/W)

**SENS_SAR_I2C_SLAVE_ADDR_10BIT_EN** Configures whether to expand the slave address to 10 bits.
0: Not expand
1: Expand
(R/W)

**SENS_SAR_I2C_REG_ADDR** Configures the register address. (R/W)

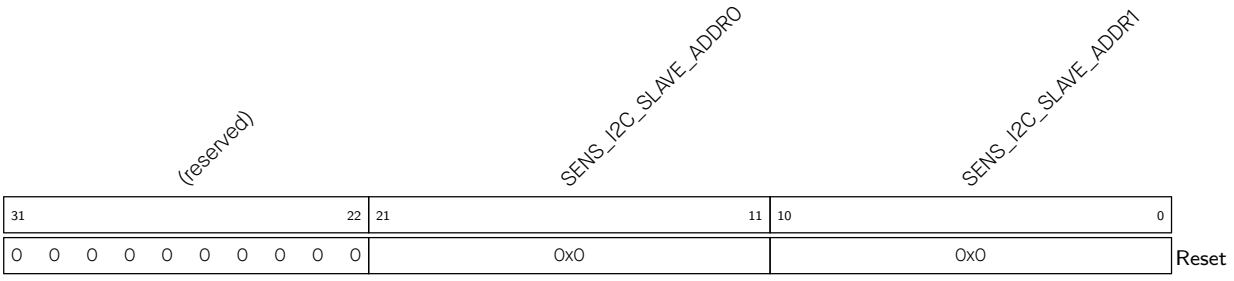
**SENS_SAR_I2C_WDATA** Configures the data to write. (R/W)

**SENS_SAR_I2C_WR_EN** Configures whether to use write command.
0: Use read command
1: Use write command
(R/W)

**SENS_SAR_I2C_START** Start RTC I2C; active only when SENS_SAR_I2C_START_FORCE = 1. (R/W)

**SENS_SAR_I2C_START_FORCE** 0: RTC I2C started by FSM; 1: RTC I2C started by software. (R/W)

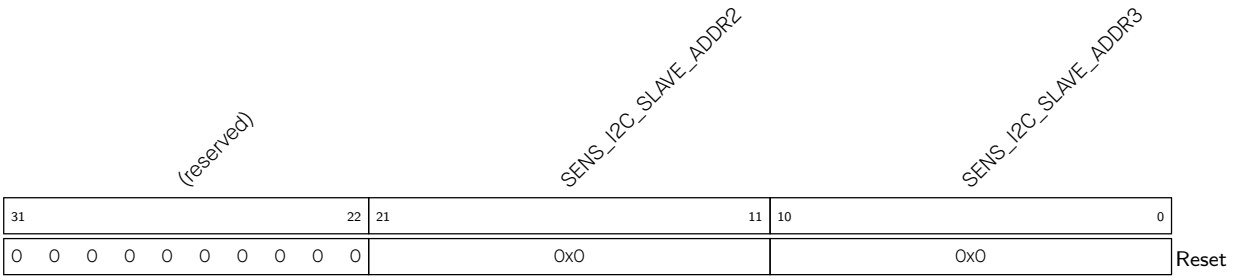
Register 2.10. SENS_SAR_SLAVE_ADDR1_REG (0x0040)




**SENS_I2C_SLAVE_ADDR1** RTC I2C slave address 1. (R/W)

**SENS_I2C_SLAVE_ADDR0** RTC I2C slave address 0. (R/W)

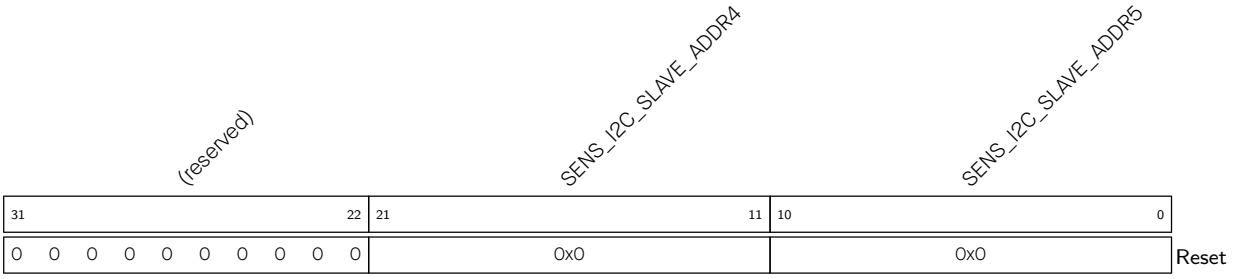
Register 2.11. SENS_SAR_SLAVE_ADDR2_REG (0x0044)




**SENS_I2C_SLAVE_ADDR3** RTC I2C slave address 3. (R/W)

**SENS_I2C_SLAVE_ADDR2** RTC I2C slave address 2. (R/W)

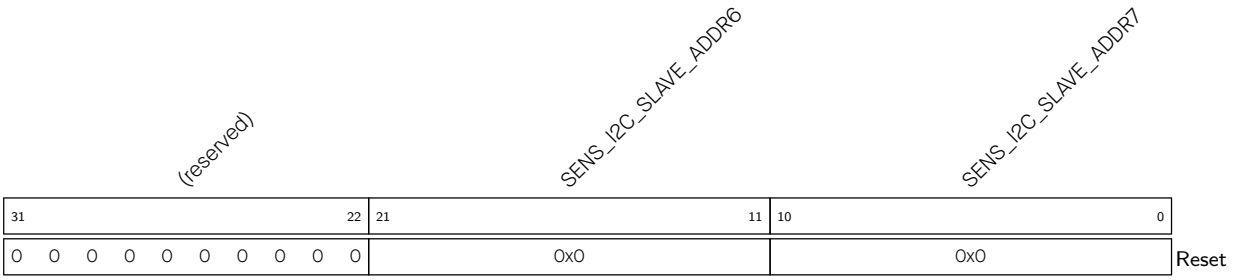
Register 2.12. SENS_SAR_SLAVE_ADDR3_REG (0x0048)




**SENS_I2C_SLAVE_ADDR5** RTC I2C slave address 5. (R/W)

**SENS_I2C_SLAVE_ADDR4** RTC I2C slave address 4. (R/W)

Register 2.13. SENS_SAR_SLAVE_ADDR4_REG (0x004C)




**SENS_I2C_SLAVE_ADDR7** RTC I2C slave address 7. (R/W)

**SENS_I2C_SLAVE_ADDR6** RTC I2C slave address 6. (R/W)

### 2.10.4 RTC I2C (I2C) Registers

The addresses in this section are relative to low-power management base address + 0x0C00 provided in Table 4.3-3 in Chapter 4 *System and Memory*.

Register 2.14. RTC_I2C_SCL_LOW_REG (0x0000)



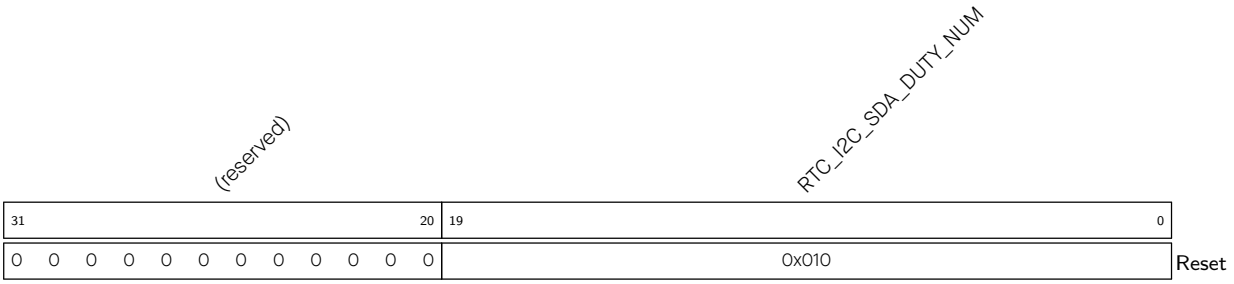

**RTC_I2C_SCL_LOW_PERIOD** This register is used to configure how many clock cycles SCL remains low. (R/W)

Register 2.15. RTC_I2C_SCL_HIGH_REG (0x0014)




**RTC_I2C_SCL_HIGH_PERIOD** This register is used to configure how many cycles SCL remains high. (R/W)

Register 2.16. RTC_I2C_SDA_DUTY_REG (0x0018)




**RTC_I2C_SDA_DUTY_NUM** The number of clock cycles between the SDA switch and the falling edge of SCL. (R/W)

Register 2.17. RTC_I2C_SCL_START_PERIOD_REG (0x001C)



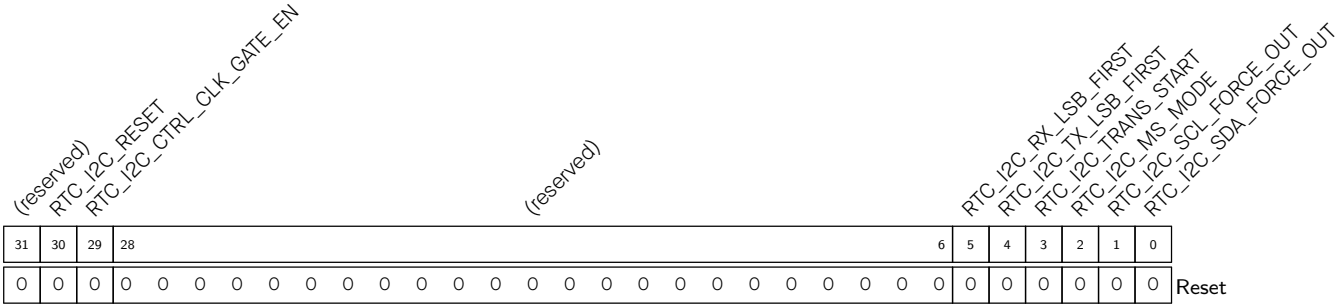

**RTC_I2C_SCL_START_PERIOD** Number of clock cycles to wait after generating a start condition. (R/W)

Register 2.18. RTC_I2C_SCL_STOP_PERIOD_REG (0x0020)

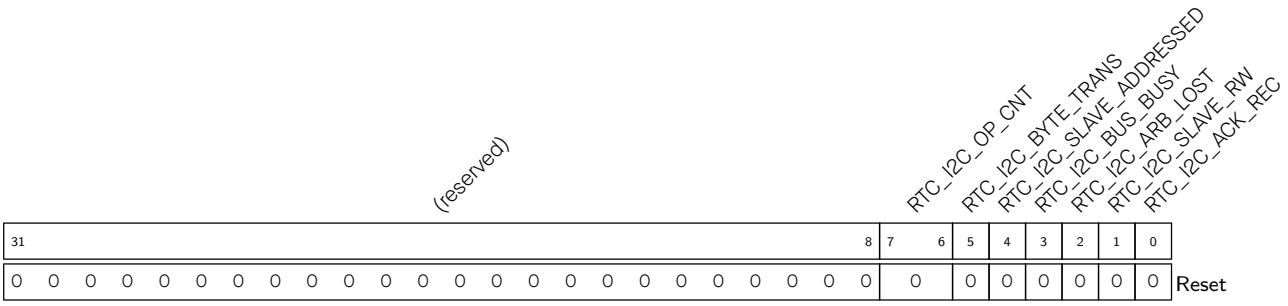



**RTC_I2C_SCL_STOP_PERIOD** Number of clock cycles to wait before generating a stop condition. (R/W)

**Register 2.19. RTC_I2C_CTRL_REG (0x0004)**


(reserved)
RTC_I2C_RESET
RTC_I2C_CTRL_CLK_GATE_EN
(reserved)
RTC_I2C_RX_LSB_FIRST
RTC_I2C_TX_LSB_FIRST
RTC_I2C_TRANS_START
RTC_I2C_MS_MODE
RTC_I2C_SCL_FORCE_OUT
RTC_I2C_SDA_FORCE_OUT
31 30 29 28 6 5 4 3 2 1 0
0 0 0 

Register 2.20. RTC_I2C_STATUS_REG (0x0008)




**RTC_I2C_ACK_REC** The received ACK value. 0: ACK; 1: NACK. (RO)

**RTC_I2C_SLAVE_RW** 0: master writes to slave; 1: master reads from slave. (RO)

**RTC_I2C_ARB_LOST** When the RTC I2C loses control of SCL line, the register changes to 1. (RO)

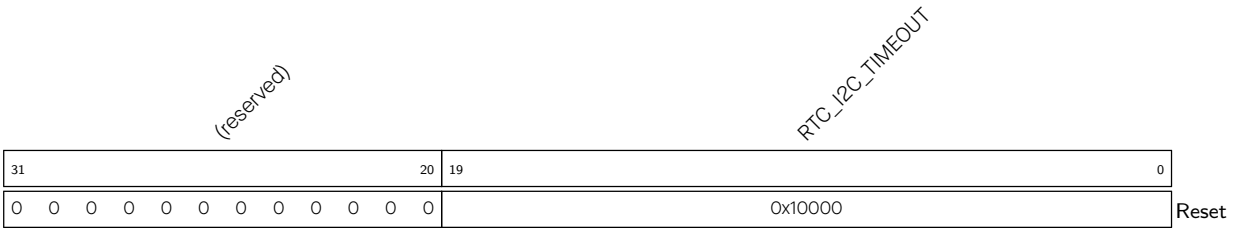
**RTC_I2C_BUS_BUSY** 0: RTC I2C bus is in idle state; 1: RTC I2C bus is busy transferring data. (RO)

**RTC_I2C_SLAVE_ADDRESSED** When the address sent by the master matches the address of the slave, then this bit will be set. (RO)

**RTC_I2C_BYTE_TRANS** This field changes to 1 when one byte is transferred. (RO)

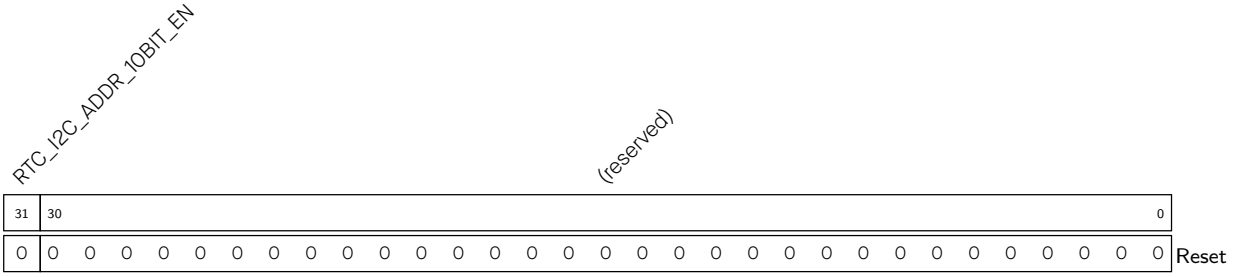
**RTC_I2C_OP_CNT** Indicate which operation is working. (RO)

Register 2.21. RTC_I2C_TIMEOUT_REG (0x000C)



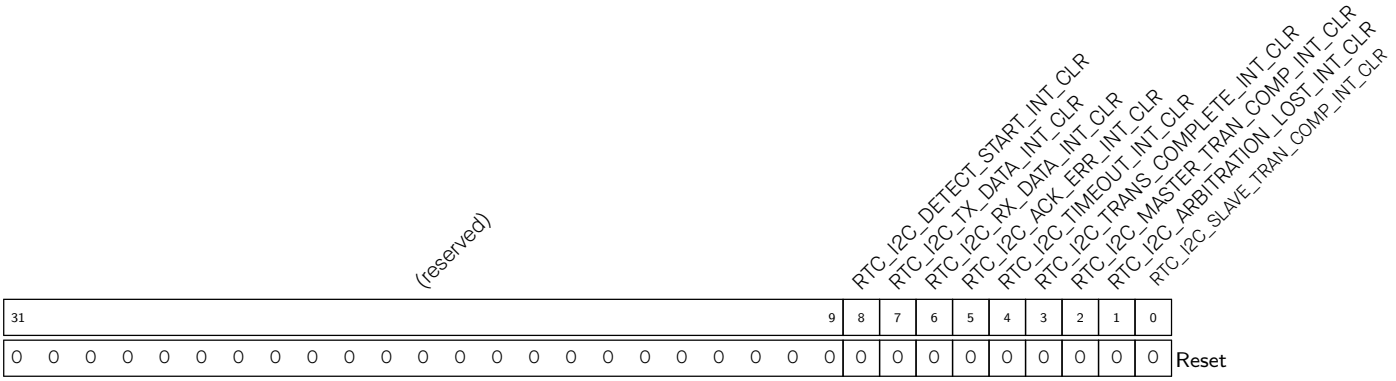

**RTC_I2C_TIMEOUT** Timeout threshold. (R/W)

Register 2.22. RTC_I2C_SLAVE_ADDR_REG (0x0010)




**RTC_I2C_ADDR_10BIT_EN** This field is used to enable the slave 10-bit addressing mode. (R/W)

Register 2.23. RTC_I2C_INT_CLR_REG (0x0024)




**RTC_I2C_SLAVE_TRAN_COMP_INT_CLR** RTC_I2C_SLAVE_TRAN_COMP_INT interrupt clear bit. (WO)

**RTC_I2C_ARBITRATION_LOST_INT_CLR** RTC_I2C_ARBITRATION_LOST_INT interrupt clear bit. (WO)

**RTC_I2C_MASTER_TRAN_COMP_INT_CLR** RTC_I2C_MASTER_TRAN_COMP_INT interrupt clear bit. (WO)

**RTC_I2C_TRANS_COMPLETE_INT_CLR** RTC_I2C_TRANS_COMPLETE_INT interrupt clear bit. (WO)

**RTC_I2C_TIMEOUT_INT_CLR** RTC_I2C_TIMEOUT_INT interrupt clear bit. (WO)

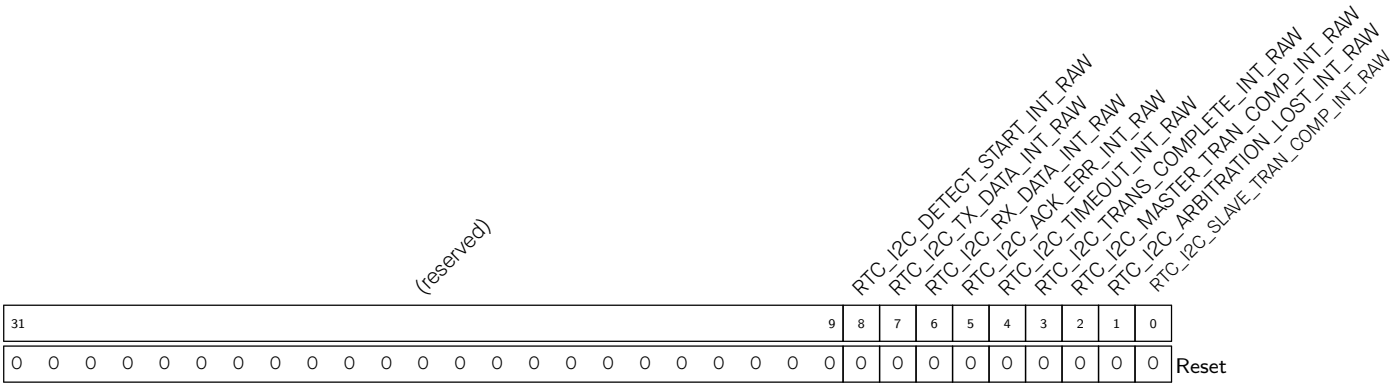
**RTC_I2C_ACK_ERR_INT_CLR** RTC_I2C_ACK_ERR_INT interrupt clear bit. (WO)

**RTC_I2C_RX_DATA_INT_CLR** RTC_I2C_RX_DATA_INT interrupt clear bit. (WO)

**RTC_I2C_TX_DATA_INT_CLR** RTC_I2C_TX_DATA_INT interrupt clear bit. (WO)

**RTC_I2C_DETECT_START_INT_CLR** RTC_I2C_DETECT_START_INT interrupt clear bit. (WO)

Register 2.24. RTC_I2C_INT_RAW_REG (0x0028)




**RTC_I2C_SLAVE_TRAN_COMP_INT_RAW** RTC_I2C_SLAVE_TRAN_COMP_INT interrupt raw bit. (RO)

**RTC_I2C_ARBITRATION_LOST_INT_RAW** RTC_I2C_ARBITRATION_LOST_INT interrupt raw bit. (RO)

**RTC_I2C_MASTER_TRAN_COMP_INT_RAW** RTC_I2C_MASTER_TRAN_COMP_INT interrupt raw bit. (RO)

**RTC_I2C_TRANS_COMPLETE_INT_RAW** RTC_I2C_TRANS_COMPLETE_INT interrupt raw bit. (RO)

**RTC_I2C_TIMEOUT_INT_RAW** RTC_I2C_TIMEOUT_INT interrupt raw bit. (RO)

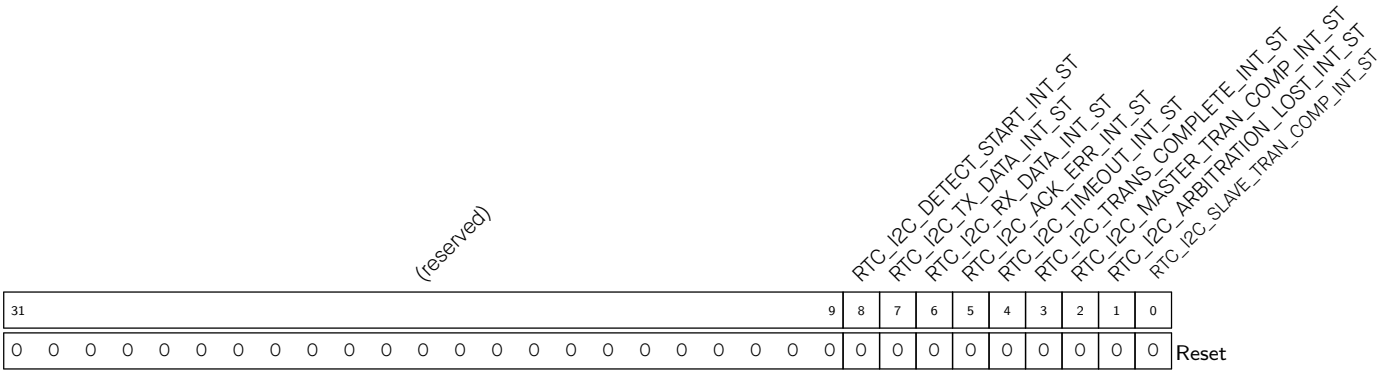
**RTC_I2C_ACK_ERR_INT_RAW** RTC_I2C_ACK_ERR_INT interrupt raw bit. (RO)

**RTC_I2C_RX_DATA_INT_RAW** RTC_I2C_RX_DATA_INT interrupt raw bit. (RO)

**RTC_I2C_TX_DATA_INT_RAW** RTC_I2C_TX_DATA_INT interrupt raw bit. (RO)

**RTC_I2C_DETECT_START_INT_RAW** RTC_I2C_DETECT_START_INT interrupt raw bit. (RO)

Register 2.25. RTC_I2C_INT_ST_REG (0x002C)




**RTC_I2C_SLAVE_TRAN_COMP_INT_ST** RTC_I2C_SLAVE_TRAN_COMP_INT interrupt status bit. (RO)

**RTC_I2C_ARBITRATION_LOST_INT_ST** RTC_I2C_ARBITRATION_LOST_INT interrupt status bit. (RO)

**RTC_I2C_MASTER_TRAN_COMP_INT_ST** RTC_I2C_MASTER_TRAN_COMP_INT interrupt status bit. (RO)

**RTC_I2C_TRANS_COMPLETE_INT_ST** RTC_I2C_TRANS_COMPLETE_INT interrupt status bit. (RO)

**RTC_I2C_TIMEOUT_INT_ST** RTC_I2C_TIMEOUT_INT interrupt status bit. (RO)

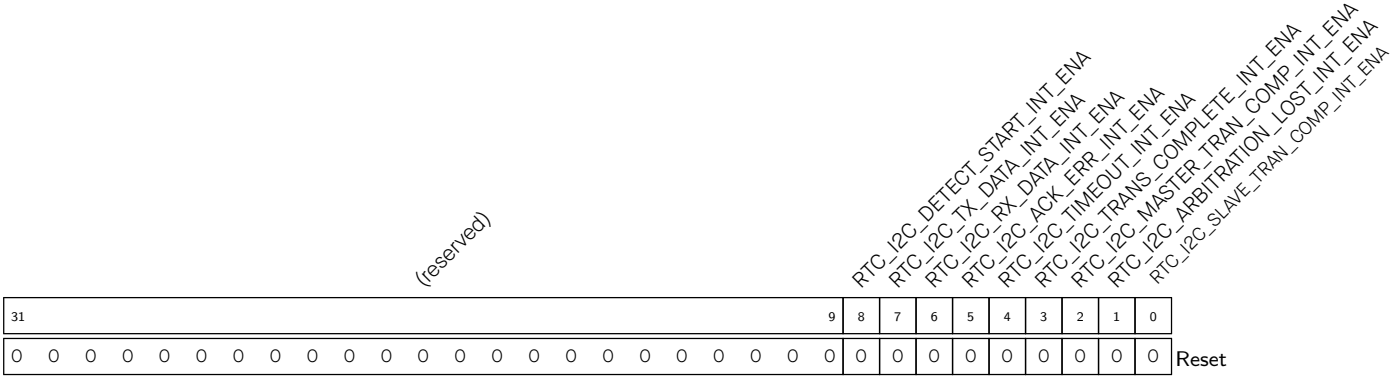
**RTC_I2C_ACK_ERR_INT_ST** RTC_I2C_ACK_ERR_INT interrupt status bit. (RO)

**RTC_I2C_RX_DATA_INT_ST** RTC_I2C_RX_DATA_INT interrupt status bit. (RO)

**RTC_I2C_TX_DATA_INT_ST** RTC_I2C_TX_DATA_INT interrupt status bit. (RO)

**RTC_I2C_DETECT_START_INT_ST** RTC_I2C_DETECT_START_INT interrupt status bit. (RO)

Register 2.26. RTC_I2C_INT_ENA_REG (0x0030)




**RTC_I2C_SLAVE_TRAN_COMP_INT_ENA** RTC_I2C_SLAVE_TRAN_COMP_INT interrupt enable bit. (R/W)

**RTC_I2C_ARBITRATION_LOST_INT_ENA** RTC_I2C_ARBITRATION_LOST_INT interrupt enable bit. (R/W)

**RTC_I2C_MASTER_TRAN_COMP_INT_ENA** RTC_I2C_MASTER_TRAN_COMP_INT interrupt enable bit. (R/W)

**RTC_I2C_TRANS_COMPLETE_INT_ENA** RTC_I2C_TRANS_COMPLETE_INT interrupt enable bit. (R/W)

**RTC_I2C_TIMEOUT_INT_ENA** RTC_I2C_TIMEOUT_INT interrupt enable bit. (R/W)

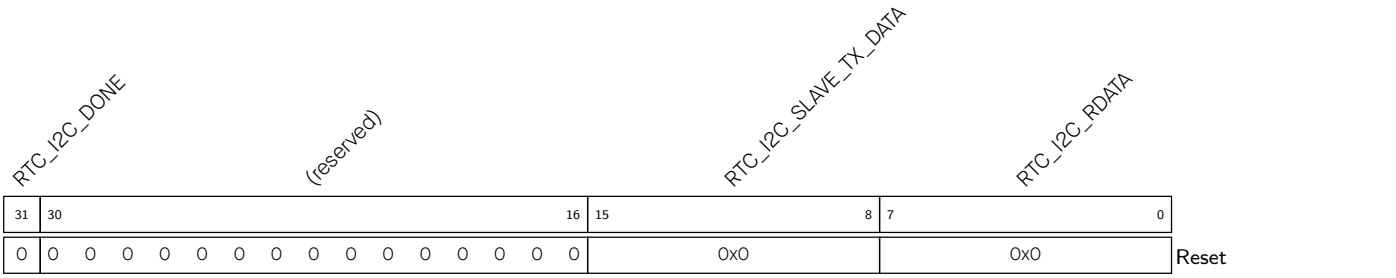
**RTC_I2C_ACK_ERR_INT_ENA** RTC_I2C_ACK_ERR_INT interrupt enable bit. (R/W)

**RTC_I2C_RX_DATA_INT_ENA** RTC_I2C_RX_DATA_INT interrupt enable bit. (R/W)

**RTC_I2C_TX_DATA_INT_ENA** RTC_I2C_TX_DATA_INT interrupt enable bit. (R/W)

**RTC_I2C_DETECT_START_INT_ENA** RTC_I2C_DETECT_START_INT interrupt enable bit. (R/W)

Register 2.27. RTC_I2C_DATA_REG (0x0034)

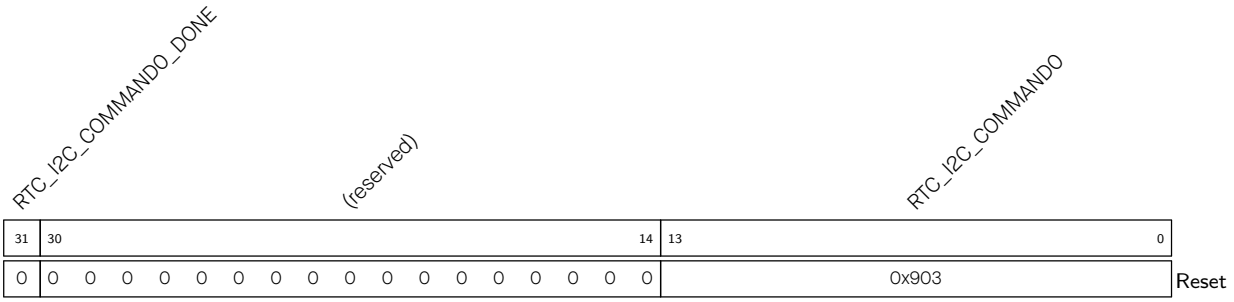



**RTC_I2C_RDATA** Data received. (RO)

**RTC_I2C_SLAVE_TX_DATA** The data sent by slave. (R/W)

**RTC_I2C_DONE** RTC I2C transmission is done. (RO)

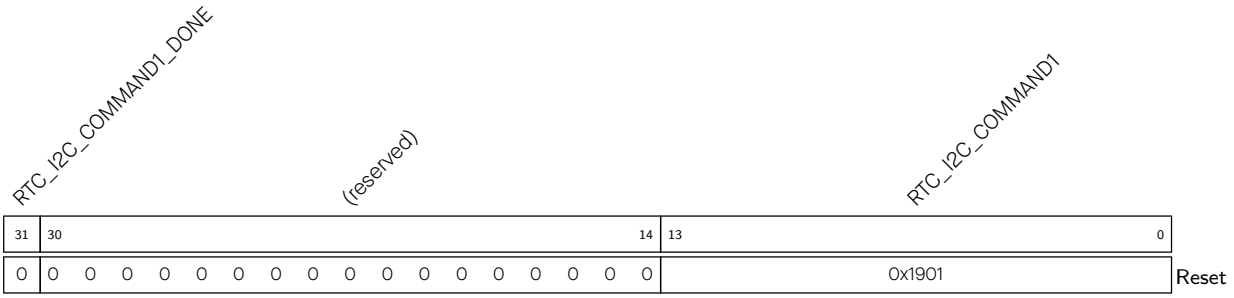
Register 2.28. RTC_I2C_CMD0_REG (0x0038)




**RTC_I2C_COMMAND0** Content of command 0. For more information, please refer to the register I2C_COMD0_REG in Chapter I2C Controller. (R/W)

**RTC_I2C_COMMAND0_DONE** When command 0 is done, this bit changes to 1. (RO)

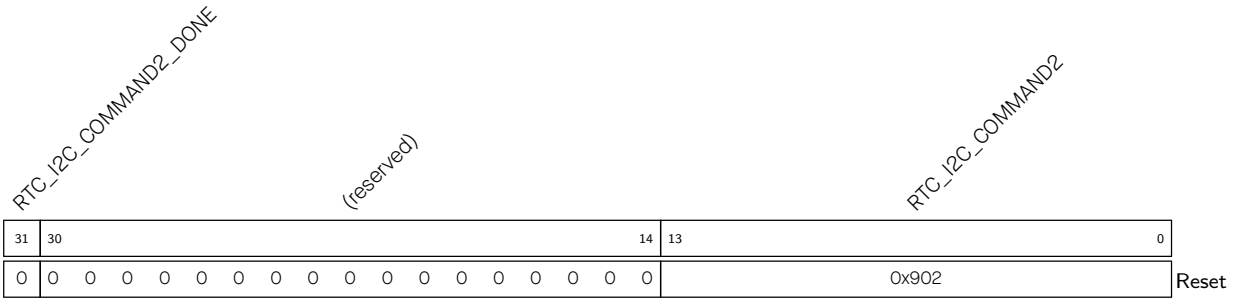
Register 2.29. RTC_I2C_CMD1_REG (0x003C)




**RTC_I2C_COMMAND1** Content of command 1. For more information, please refer to the register I2C_COMD1_REG in Chapter I2C Controller. (R/W)

**RTC_I2C_COMMAND1_DONE** When command 1 is done, this bit changes to 1. (RO)

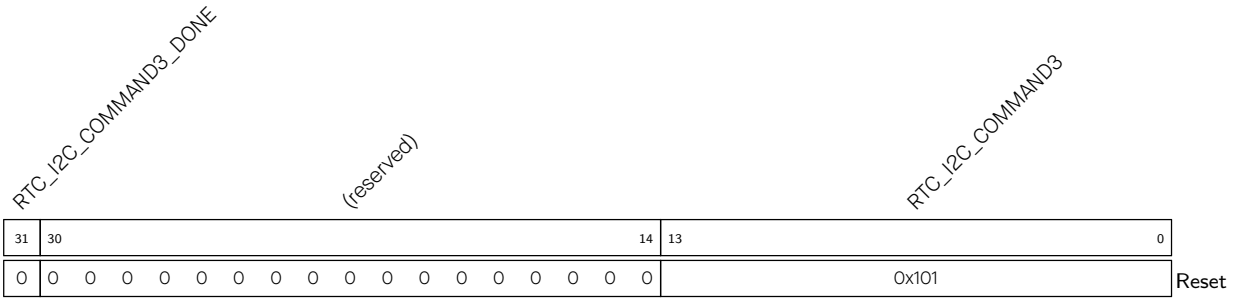
Register 2.30. RTC_I2C_CMD2_REG (0x0040)




**RTC_I2C_COMMAND2** Content of command 2. For more information, please refer to the register I2C_COMD2_REG in Chapter I2C Controller. (R/W)

**RTC_I2C_COMMAND2_DONE** When command 2 is done, this bit changes to 1. (RO)

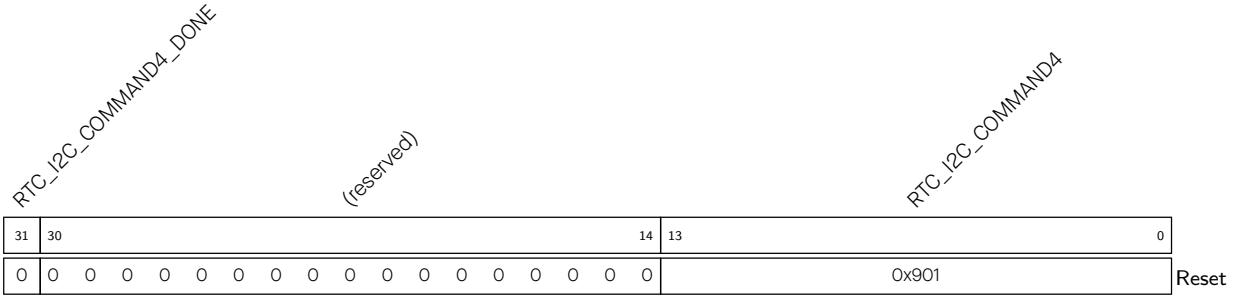
Register 2.31. RTC_I2C_CMD3_REG (0x0044)




**RTC_I2C_COMMAND3** Content of command 3. For more information, please refer to the register I2C_COMD3_REG in Chapter I2C Controller. (R/W)

**RTC_I2C_COMMAND3_DONE** When command 3 is done, this bit changes to 1. (RO)

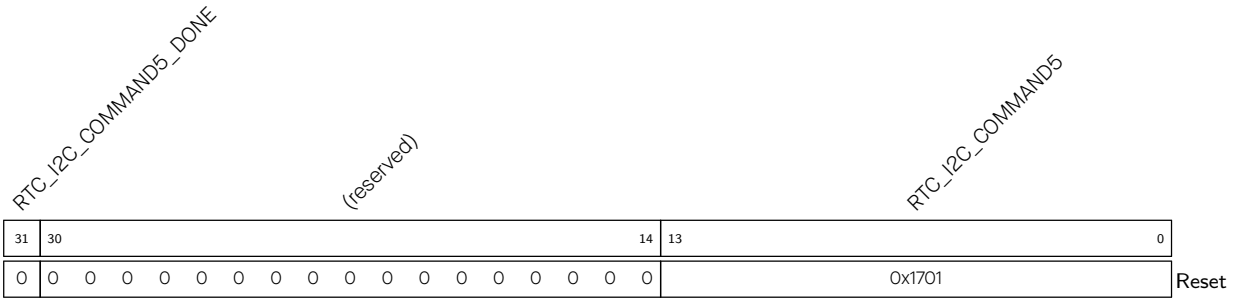
Register 2.32. RTC_I2C_CMD4_REG (0x0048)




**RTC_I2C_COMMAND4** Content of command 4. For more information, please refer to the register I2C_COMD4_REG in Chapter I2C Controller. (R/W)

**RTC_I2C_COMMAND4_DONE** When command 4 is done, this bit changes to 1. (RO)

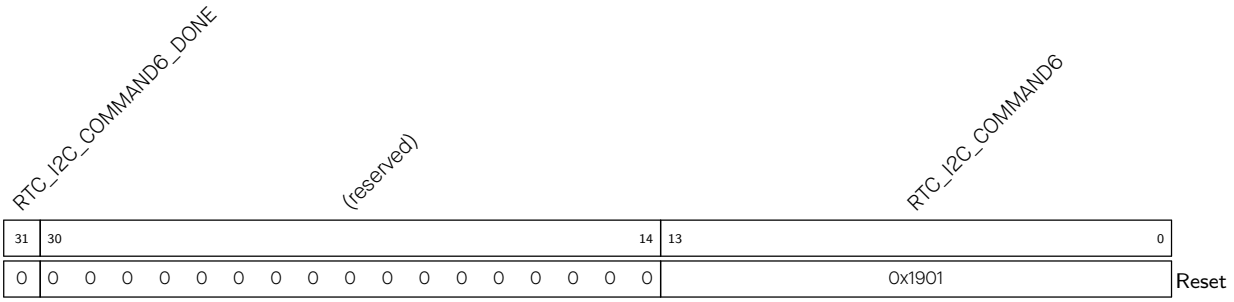
Register 2.33. RTC_I2C_CMD5_REG (0x004C)




**RTC_I2C_COMMAND5** Content of command 5. For more information, please refer to the register I2C_COMD5_REG in Chapter I2C Controller. (R/W)

**RTC_I2C_COMMAND5_DONE** When command 5 is done, this bit changes to 1. (RO)

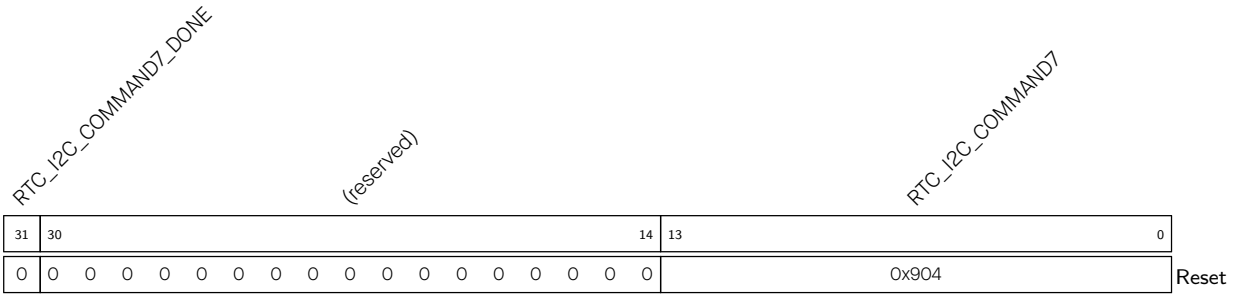
Register 2.34. RTC_I2C_CMD6_REG (0x0050)




**RTC_I2C_COMMAND6** Content of command 6. For more information, please refer to the register I2C_COMD6_REG in Chapter I2C Controller. (R/W)

**RTC_I2C_COMMAND6_DONE** When command 6 is done, this bit changes to 1. (RO)

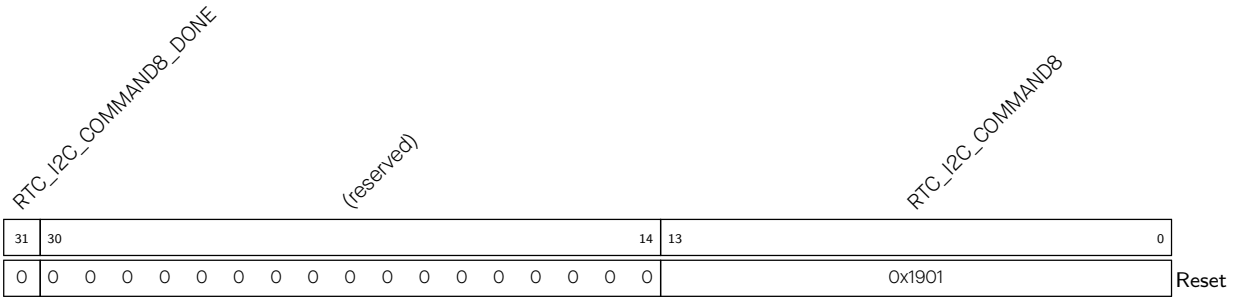
Register 2.35. RTC_I2C_CMD7_REG (0x0054)




**RTC_I2C_COMMAND7** Content of command 7. For more information, please refer to the register I2C_COMD7_REG in Chapter I2C Controller. (R/W)

**RTC_I2C_COMMAND7_DONE** When command 7 is done, this bit changes to 1. (RO)

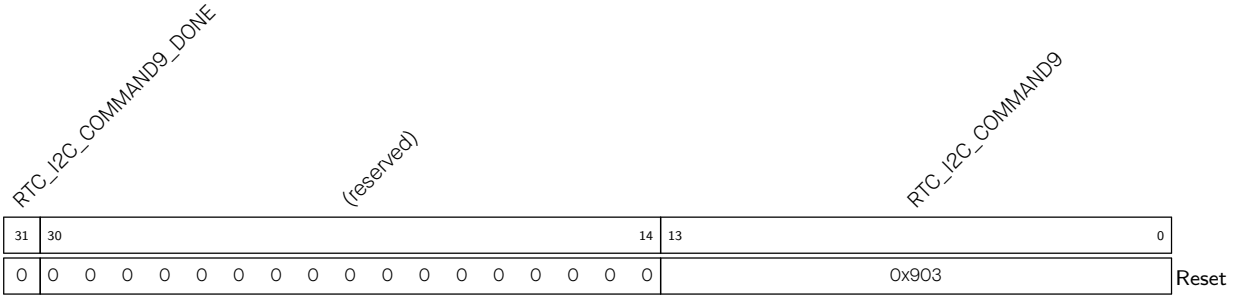
Register 2.36. RTC_I2C_CMD8_REG (0x0058)




**RTC_I2C_COMMAND8** Content of command 8. For more information, please refer to the register I2C_COMD8_REG in Chapter I2C Controller. (R/W)

**RTC_I2C_COMMAND8_DONE** When command 8 is done, this bit changes to 1. (RO)

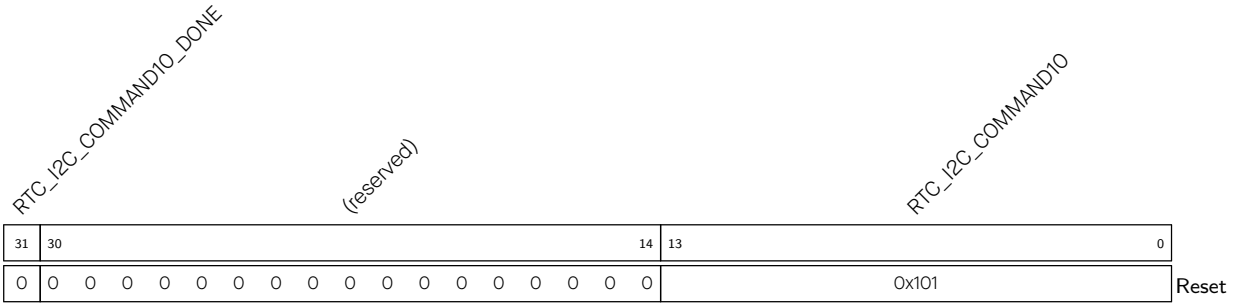
Register 2.37. RTC_I2C_CMD9_REG (0x005C)




**RTC_I2C_COMMAND9** Content of command 9. For more information, please refer to the register I2C_COMD9_REG in Chapter I2C Controller. (R/W)

**RTC_I2C_COMMAND9_DONE** When command 9 is done, this bit changes to 1. (RO)

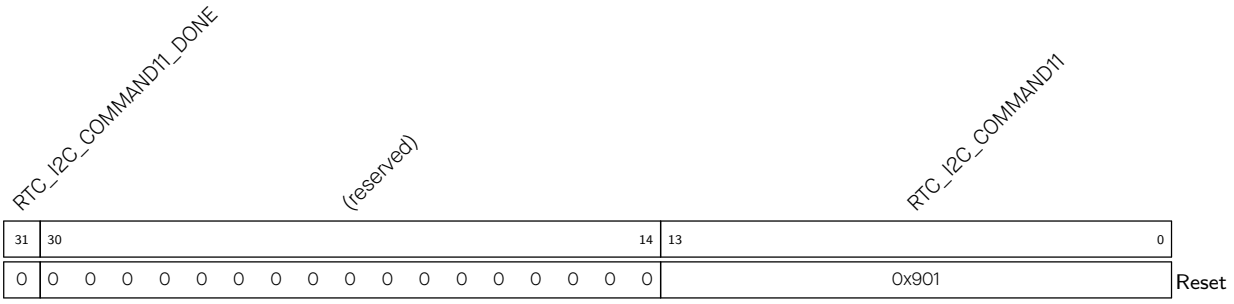
Register 2.38. RTC_I2C_CMD10_REG (0x0060)




**RTC_I2C_COMMAND10** Content of command 10. For more information, please refer to the register I2C_COMD10_REG in Chapter I2C Controller. (R/W)

**RTC_I2C_COMMAND10_DONE** When command 10 is done, this bit changes to 1. (RO)

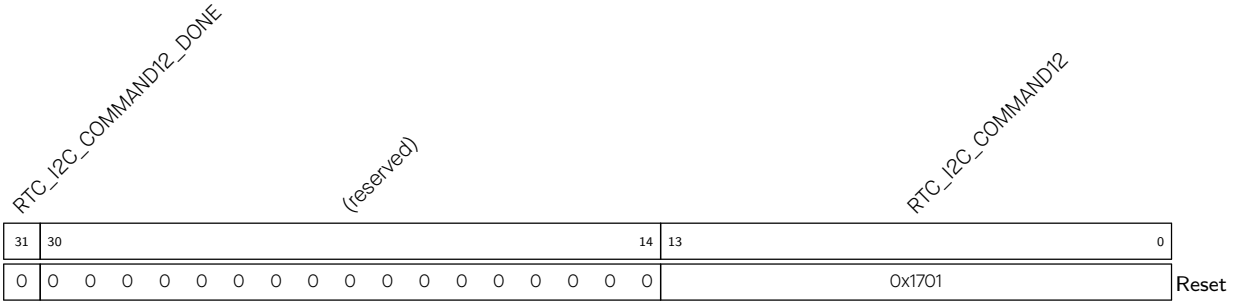
Register 2.39. RTC_I2C_CMD11_REG (0x0064)




**RTC_I2C_COMMAND11** Content of command 11. For more information, please refer to the register I2C_COMD11_REG in Chapter I2C Controller. (R/W)

**RTC_I2C_COMMAND11_DONE** When command 11 is done, this bit changes to 1. (RO)

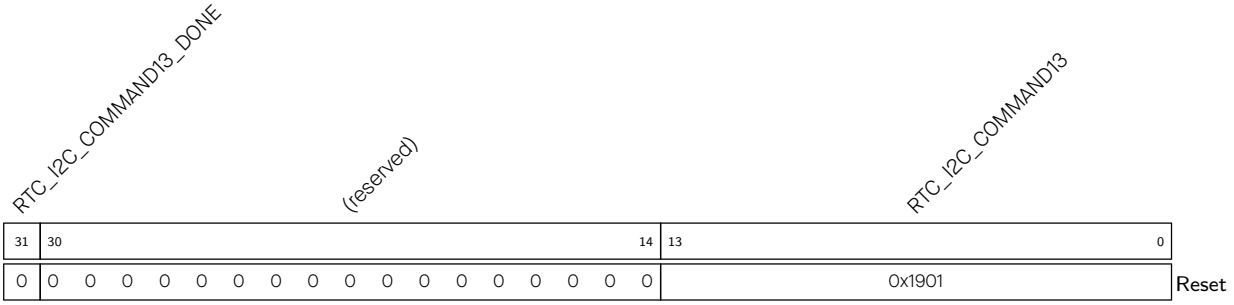
Register 2.40. RTC_I2C_CMD12_REG (0x0068)




**RTC_I2C_COMMAND12** Content of command 12. For more information, please refer to the register I2C_COMD12_REG in Chapter I2C Controller. (R/W)

**RTC_I2C_COMMAND12_DONE** When command 12 is done, this bit changes to 1. (RO)

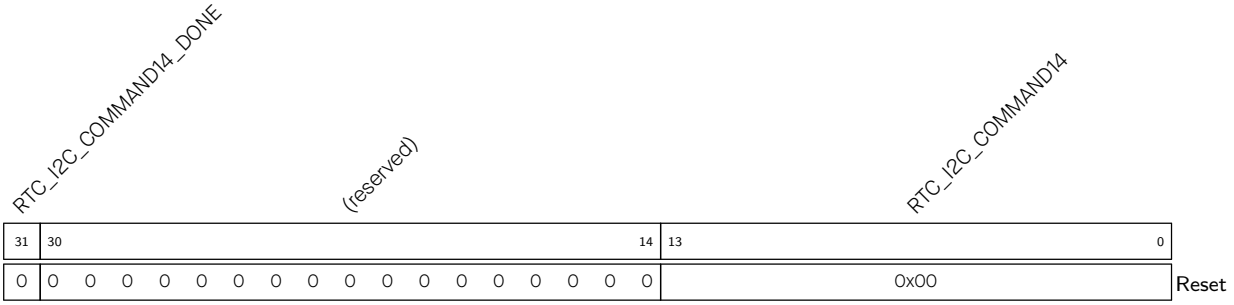
Register 2.41. RTC_I2C_CMD13_REG (0x006C)




**RTC_I2C_COMMAND13** Content of command 13. For more information, please refer to the register I2C_COMD13_REG in Chapter I2C Controller. (R/W)

**RTC_I2C_COMMAND13_DONE** When command 13 is done, this bit changes to 1. (RO)

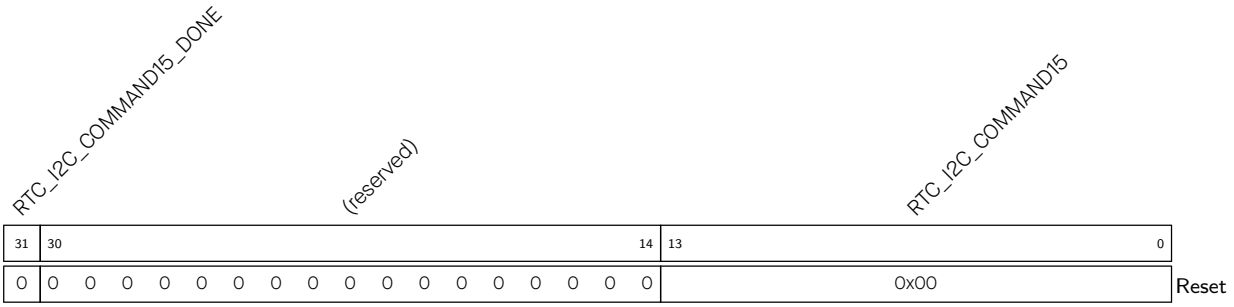
Register 2.42. RTC_I2C_CMD14_REG (0x0070)




**RTC_I2C_COMMAND14** Content of command 14. For more information, please refer to the register I2C_COMD14_REG in Chapter I2C Controller. (R/W)

**RTC_I2C_COMMAND14_DONE** When command 14 is done, this bit changes to 1. (RO)

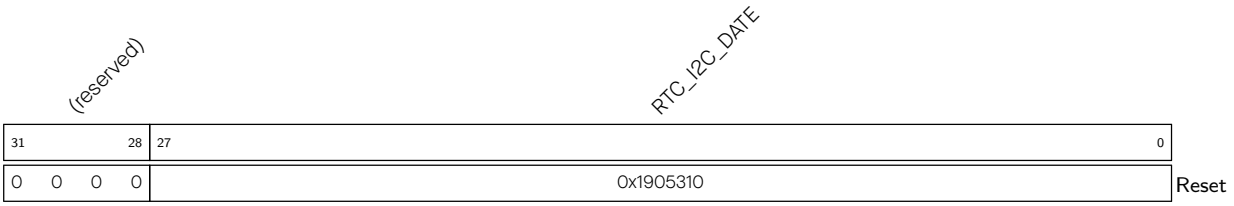
Register 2.43. RTC_I2C_CMD15_REG (0x0074)




**RTC_I2C_COMMAND15** Content of command 15. For more information, please refer to the register I2C_COMD15_REG in Chapter I2C Controller. (R/W)

**RTC_I2C_COMMAND15_DONE** When command 15 is done, this bit changes to 1. (RO)

Register 2.44. RTC_I2C_DATE_REG (0x00FC)




**RTC_I2C_DATE** Version control register (R/W)